

Rockchip 多目拼接产线标定指南

ID: RK-YH-SF-005

发布版本: V1.2.3

日期: 2025.02.12

文件密级: ☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供, 瑞芯微电子股份有限公司(“本公司”, 下同) 不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因, 本文档将可能在未经任何通知的情况下, 不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标, 归本公司所有。

本文档可能提及的其他所有注册商标或商标, 由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴, 非经本公司书面许可, 任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部, 并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址: 福建省福州市铜盘路软件园A区18号

网址: www.rock-chips.com

客户服务电话: +86-4007-700-590

客户服务传真: +86-591-83951833

客户服务邮箱: fae@rock-chips.com

前言

本文面向应用双目拼接方案的开发工程师，主要介绍双目拼接产线标定基本原理、操作步骤及使用注意事项等。

产品版本

与本文档相对应的产品版本如下：

产品名称	产品版本
RV1103B	
RV1106B	
RV1106	
RV1126	
RK3588	
RK3576	

修订记录

日期	版本	作者	修改说明
2023.03.07	V0.1.1	mingpei.liu	初始版本
2023.03.15	V0.1.2	mingpei.liu	完善场景布局说明
2023.04.18	V0.1.3	mingpei.liu	添加标定异常分析
2023.06.06	V0.1.4	mingpei.liu	完善标定异常分析
2023.06.20	V1.1.1	mingpei.liu	引入模型标定文件引导标定
2023.07.10	V1.1.2	mingpei.liu	完善拍图注意事项
2023.11.29	V1.1.3	mingpei.liu	完善拍图注意事项
2023.12.08	V1.1.4	mingpei.liu	完善模组设计说明
2024.04.25	V1.2.0	mingpei.liu	添加多目产线标定说明
2024.05.31	V1.2.1	mingpei.liu	添加产线验证主观评测说明
2024.09.29	V1.2.2	mingpei.liu	添加产线标定输出参数说明
2025.02.12	V1.2.3	mingpei.liu	修改部分用词

目录

Rockchip 多目拼接产线标定指南

1. 概述
 - 1.1 多目拼接
 - 1.2 产线标定
 - 1.3 产线标定输出文件
 - 1.3.1 标定参数
 - 1.3.2 卡控参数
 - 1.3.3 标定Log
 - 1.3.4 输出图
2. 模组设计要求
 - 2.1 模组设计指标
 - 2.1.1 双目模组结构
 - 2.1.2 镜头选型
 - 2.2 模组设计实例
3. 双目产线标定
 - 3.1 操作环境
 - 3.1.1 操作环境设计
 - 3.1.2 操作环境实例
 - 3.2 标定场景布局定义
 - 3.3 棋盘格规格设计
 - 3.4 操作步骤
 - 3.5 API接口及Demo
 - 3.5.1 接口定义
 - 3.5.2 标定Demo
4. 双目产线验证
 - 4.1 客观评测
 - 4.1.1 操作环境
 - 4.1.2 棋盘格规格设计
 - 4.1.3 API接口及Demo
 - 4.1.3.1 接口定义
 - 4.1.3.2 验证Demo
 - 4.2 主观评测
5. 多目产线标定
 - 5.1 便捷模式
 - 5.1.1 拍摄场景
 - 5.1.2 拍图流程
 - 5.2 快速模式
 - 5.2.1 标定场景
 - 5.2.2 拍图流程
 - 5.3 API接口及Demo
 - 5.3.1 接口定义
 - 5.3.2 标定Demo
6. 多目产线验证
 - 6.1 客观评测
 - 6.1.1 便捷模式
 - 6.1.2 快速模式
 - 6.1.3 API接口及Demo
 - 6.2 主观评测
7. 标定异常分析
 - 7.1 RK_AVS_CALIB_STATUS_INVALID_PARAM
 - 7.2 RK_AVS_CALIB_STATUS_EOF
 - 7.2.1 Split chessboard error
 - 7.2.2 Good chessboard number < n

7.3 标定精度低

7.3.1 问题分析

7.3.2 错误示例

1. 概述

1.1 多目拼接

多目拼接可以将多路输入图像拼接为一路图像，从而获取更大视野的图像。

1.2 产线标定

多目拼接产线标定通过对所提供棋盘格模具的单张拍摄，借助模型标定的标定结果作为种子文件来获得立体相机模组的所有标定参数，同时返回所得标定参数的相对误差。标定参数包括左右相机的内参、左右相机的相对外参。内参包括焦距 (f_x , f_y)，主点位置 (u_0 , v_0) 以及光学失真系数 (k_1 , k_2 , k_3 , p_1 , p_2)；相对外参包括三轴角度 *Tilt*, *Pan*, *Roll*, x 轴 y 轴 z 轴的平移参数。

1.3 产线标定输出文件

1.3.1 标定参数

产线标定的xml文件为多目相机标定参数，包括：相机个数、标定图像分辨率、相机成像模型、镜头水平FOV、相机的内参、相邻相机之间的外参和标定内外参数的误差。

1.3.2 卡控参数

输出的productCheckParams.xml为标定卡控参数，可用于检查标定是否成功，其参数的意义如下：

- leftInternalRms: 左相机内参误差（像素）
- rightInternalRms: 右相机内参误差（像素）
- externalRms: 两个相机间的外参误差（像素）
- euler_angles_x: 两个相机x方向的相对角度(度)
- euler_angles_y: 两个相机y方向的相对角度(度)
- euler_angles_z: 两个相机z方向的相对角度(度)
- t_x: 两个相机x方向的相对距离（mm）
- t_y: 两个相机y方向的相对距离（mm）
- t_z: 两个相机z方向的相对距离（mm）

通过限定卡控参数，筛选出标定不合格的模组。卡控参数的阈值需根据工厂试产的大数据设置一个合理值，一般卡控内外参数的标定误差（leftInternalRms、rightInternalRms、externalRms）。其它参数用于描述模组的结构信息，可用于反向验证模组组装的一致性，一般不需要卡控。

1.3.3 标定Log

输出的rkCalibLog.txt文件为产线标定算法内部输出的log，用于排查标定失败原因。

1.3.4 输出图

输出图包括棋盘格角点输出图和拼接mask图，其中棋盘格角点输出图用于查看棋盘格拍摄环境是否符合需求，角点提取是否准确；拼接mask图用于查看该标定参数作用于模组拼接时的畸变矫正是否符合预期以及每个相机图像在输出图上的位置。

2. 模组设计要求

模组设计从需求出发，根据需求进行sensor和镜头的选型，并设计合适的光轴夹角。文档以双目模组为例，介绍模组设计的基本要求。

2.1 模组设计指标

2.1.1 双目模组结构

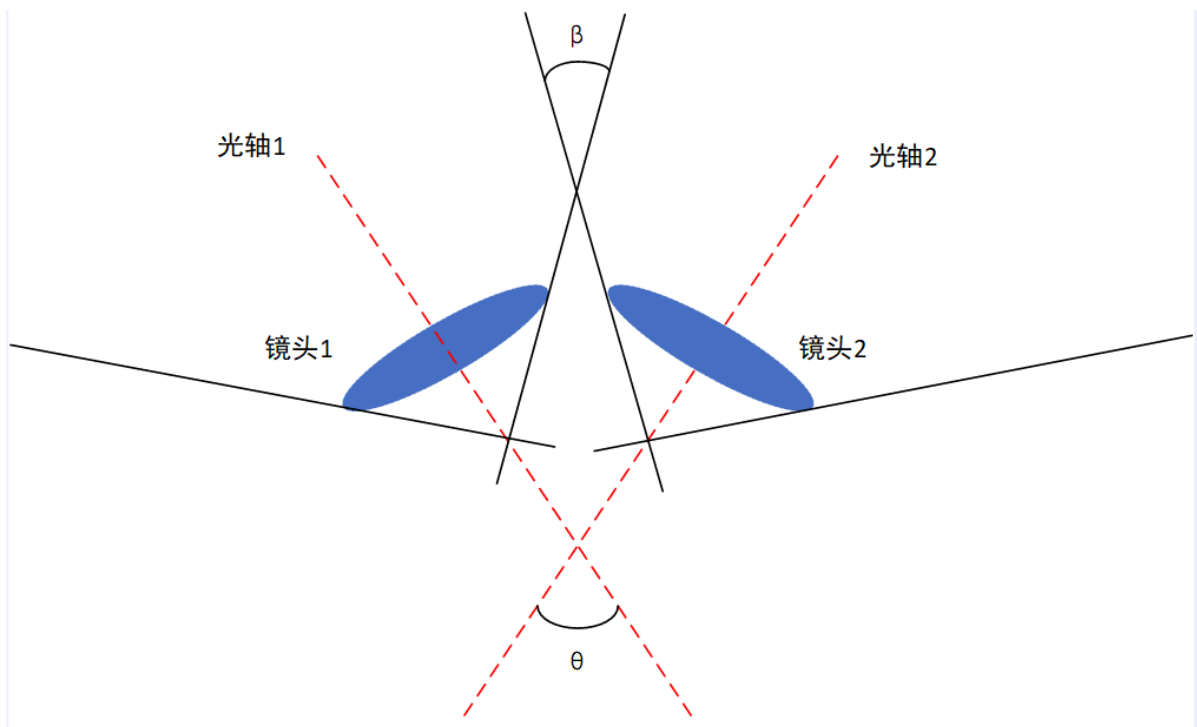


图2-1 双目模组设计示意图

具体设计要求如下：

- 两个相机光轴之间水平夹角为 θ 。
- 两个相机光轴之间垂直夹角为 0° 。
- 两个相机水平方向重叠区域的FOV为 β ($\beta > 15^\circ$)。

- 要求工厂组装时组装精度为 $\pm 3^\circ$ ，既水平夹角为 $\theta \pm 3^\circ$ ，垂直夹角为 $\pm 3^\circ$ 。

相机光轴之间的水平夹角 θ 可根据需求进行调整： θ 越大重叠区域越小，拼接输出FOV越大； θ 越小重叠区域越大，拼接输出FOV越小。当光轴之间的水平夹角 θ 固定时：镜头的FOV与拼接输出的FOV越大；镜头的FOV越小，拼接输出的FOV越小。

如图2-2所示，为双目110度拼接170度的参考设计示意图。

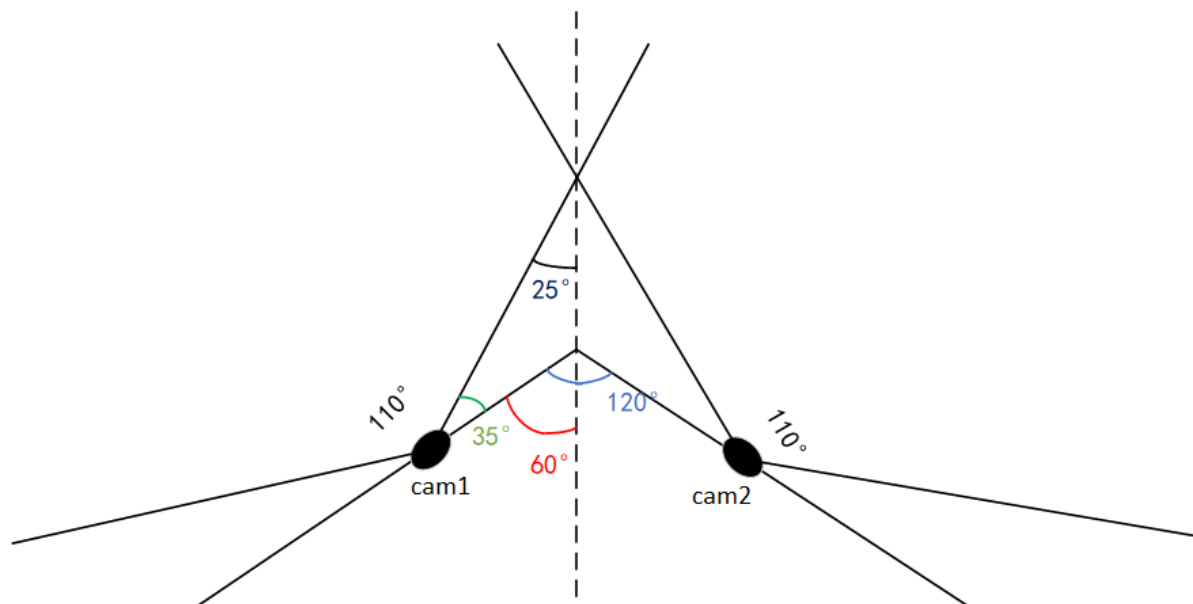


图2-2 双目110度参考设计示意图

2.1.2 镜头选型

镜头选型选择普通的广角镜头，不支持经过特殊处理过的镜头，推荐使用基于equidistant模型设计的广角镜头。理想情况下实际像高与入射角度成线性关系，如图2-3所示。

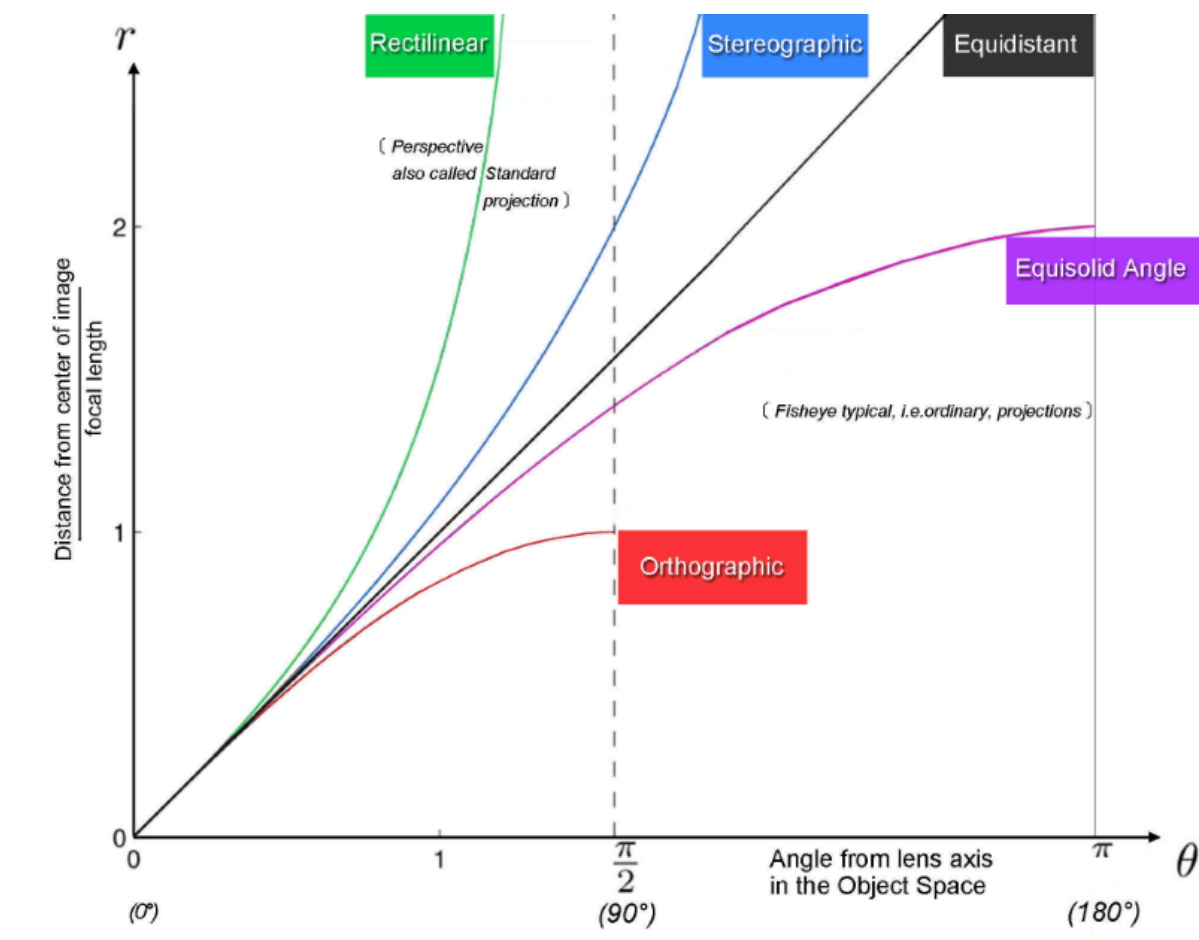


图2-3 镜头类型

如图表2-1所示，为某款镜头的畸变映射表

Y Angle (deg)	Tan Shift	Sag Shift	Real Height	Ref. Height	Distortion
0	-0.00049391	-0.00049391	0	0	0.00%
0.65000175	-0.00049655	-0.00050008	0.03563319	0.03563393	0.00%
1.30000349	-0.00050524	-0.00051849	0.07127108	0.07127704	-0.01%
1.95000524	-0.00051835	-0.00054872	0.10691836	0.10693851	-0.02%
2.60000699	-0.00053319	-0.00059014	0.14257963	0.14262753	-0.03%
3.25000874	-0.00054613	-0.00064186	0.17825943	0.17835335	-0.05%
3.90001048	-0.00055269	-0.00070279	0.21396217	0.21412523	-0.08%
4.55001223	-0.00054768	-0.0007716	0.24969213	0.24995249	-0.10%
5.20001398	-0.0005254	-0.00084683	0.28545344	0.2858445	-0.14%
5.85001573	-0.00047979	-0.00092684	0.32125002	0.3218107	-0.17%
6.50001747	-0.00040467	-0.00100992	0.3570856	0.35786061	-0.22%
7.15001922	-0.00029394	-0.00109424	0.39296371	0.39400384	-0.26%
7.80002097	-0.00014183	-0.00117797	0.42888763	0.43025009	-0.32%
8.45002272	0.0000569	-0.00125927	0.4648604	0.46660918	-0.37%
9.10002446	0.00030673	-0.00133634	0.50088481	0.50309104	-0.44%
9.75002621	0.00061116	-0.00140747	0.5369634	0.53970573	-0.51%
10.40002796	0.00097253	-0.00147107	0.57309846	0.57646346	-0.58%
11.05002971	0.00139185	-0.0015257	0.60929201	0.61337458	-0.67%
11.70003145	0.00186864	-0.00157014	0.64554582	0.65044962	-0.75%
12.3500332	0.00240087	-0.00160336	0.6818614	0.68769929	-0.85%
13.00003495	0.00298487	-0.0016246	0.71824001	0.72513447	-0.95%
13.6500367	0.0036153	-0.00163337	0.75468268	0.76276628	-1.06%
14.30003844	0.00428519	-0.00162949	0.79119017	0.80060602	-1.18%
14.95004019	0.004986	-0.00161306	0.82776301	0.83866525	-1.30%
15.60004194	0.00570772	-0.00158449	0.86440149	0.87695577	-1.43%
16.25004369	0.00643905	-0.00154453	0.90110566	0.91548964	-1.57%
16.90004543	0.00716756	-0.0014942	0.93787531	0.9542792	-1.72%
17.55004718	0.00787996	-0.00143484	0.97470999	0.9933371	-1.88%
18.20004893	0.00856233	-0.00136806	1.01160899	1.03267629	-2.04%
18.85005068	0.00920042	-0.00129572	1.04857129	1.07231006	-2.21%
19.50005242	0.00977997	-0.00121991	1.08559562	1.11225204	-2.40%
20.15005417	0.01028699	-0.00114292	1.12268035	1.15251626	-2.59%

Y Angle (deg)	Tan Shift	Sag Shift	Real Height	Ref. Height	Distortion
20.80005592	0.01070813	-0.00106719	1.15982353	1.19311711	-2.79%
21.45005767	0.01103095	-0.00099531	1.19702281	1.23406944	-3.00%
22.10005941	0.01124423	-0.00092993	1.23427545	1.27538849	-3.22%
22.75006116	0.01133829	-0.00087375	1.27157827	1.31709	-3.46%
23.40006291	0.0113052	-0.00082945	1.30892758	1.35919019	-3.70%
24.05006466	0.01113902	-0.00079969	1.3463192	1.4017058	-3.95%
24.7000664	0.01083597	-0.00078704	1.38374837	1.44465412	-4.22%
25.35006815	0.01039456	-0.00079392	1.42120972	1.48805302	-4.49%
26.0000699	0.00981568	-0.00082261	1.45869726	1.53192098	-4.78%
26.65007165	0.00910262	-0.00087518	1.49620433	1.57627711	-5.08%
27.30007339	0.00826099	-0.00095345	1.53372356	1.62114124	-5.39%
27.95007514	0.0072987	-0.00105902	1.57124684	1.66653388	-5.72%
28.60007689	0.00622577	-0.00119316	1.60876534	1.71247633	-6.06%
29.25007864	0.00505412	-0.00135687	1.64626947	1.75899067	-6.41%
29.90008038	0.00379732	-0.00155081	1.68374887	1.80609987	-6.77%
30.55008213	0.00247028	-0.00177534	1.72119244	1.85382776	-7.15%
31.20008388	0.00108893	-0.00203047	1.75858837	1.90219915	-7.55%
31.85008563	-0.0003302	-0.00231591	1.79592412	1.95123984	-7.96%
32.50008737	-0.00177032	-0.00263104	1.83318649	2.00097673	-8.39%
33.15008912	-0.00321482	-0.00297495	1.8703617	2.05143781	-8.83%
33.80009087	-0.00464766	-0.00334644	1.90743537	2.1026523	-9.28%
34.45009262	-0.00605376	-0.00374405	1.94439266	2.1546507	-9.76%
35.10009436	-0.00741937	-0.00416611	1.98121833	2.20746482	-10.25%
35.75009611	-0.00873238	-0.00461073	2.0178968	2.26112794	-10.76%
36.40009786	-0.00998263	-0.00507584	2.05441223	2.31567486	-11.28%
37.05009961	-0.01116211	-0.00555925	2.09074867	2.37114197	-11.83%
37.70010135	-0.01226512	-0.00605866	2.12689009	2.42756742	-12.39%
38.3501031	-0.01328839	-0.0065717	2.1628205	2.48499117	-12.96%
39.00010485	-0.01423106	-0.00709595	2.19852403	2.54345512	-13.56%
39.6501066	-0.01509469	-0.00762901	2.23398503	2.60300326	-14.18%
40.30010834	-0.01588307	-0.00816847	2.26918812	2.66368181	-14.81%
40.95011009	-0.01660204	-0.00871197	2.30411829	2.72553933	-15.46%

Y Angle (deg)	Tan Shift	Sag Shift	Real Height	Ref. Height	Distortion
41.60011184	-0.01725924	-0.00925722	2.33876092	2.78862691	-16.13%
42.25011359	-0.01786379	-0.00980198	2.37310188	2.85299836	-16.82%
42.90011533	-0.01842585	-0.01034412	2.40712752	2.91871036	-17.53%
43.55011708	-0.01895627	-0.01088159	2.44082473	2.98582266	-18.25%
44.20011883	-0.01946602	-0.01141243	2.47418093	3.05439837	-19.00%
44.85012058	-0.01996576	-0.01193475	2.50718408	3.12450411	-19.76%
45.50012232	-0.02046524	-0.01244676	2.53982267	3.19621034	-20.54%
46.15012407	-0.02097281	-0.01294669	2.57208567	3.26959159	-21.33%
46.80012582	-0.02149479	-0.01343283	2.60396253	3.34472683	-22.15%
47.45012757	-0.02203495	-0.01390346	2.63544313	3.42169975	-22.98%
48.10012931	-0.02259393	-0.01435686	2.66651766	3.50059917	-23.83%
48.75013106	-0.02316871	-0.01479124	2.69717666	3.58151942	-24.69%
49.40013281	-0.02375206	-0.01520474	2.72741087	3.66456079	-25.57%
50.05013456	-0.02433211	-0.0155954	2.75721118	3.74983004	-26.47%
50.7001363	-0.02489187	-0.01596109	2.78656855	3.8374409	-27.38%
51.35013805	-0.02540891	-0.01629948	2.81547394	3.92751467	-28.31%
52.0001398	-0.02585501	-0.01660808	2.84391821	4.02018088	-29.26%
52.65014155	-0.02619599	-0.0168841	2.87189206	4.11557799	-30.22%
53.30014329	-0.02639166	-0.01712453	2.89938598	4.21385421	-31.19%
53.95014504	-0.02639578	-0.01732603	2.92639013	4.31516836	-32.18%
54.60014679	-0.02615637	-0.01748501	2.95289434	4.41969082	-33.19%
55.25014854	-0.02561598	-0.01759754	2.97888808	4.52760467	-34.21%
55.90015028	-0.02471236	-0.01765942	3.00436037	4.63910687	-35.24%
56.55015203	-0.02337914	-0.01766614	3.02929988	4.7544096	-36.28%
57.20015378	-0.02154691	-0.01761297	3.05369486	4.87374179	-37.34%
57.85015553	-0.01914437	-0.01749492	3.07753321	4.9973508	-38.42%
58.50015727	-0.01609977	-0.01730688	3.10080256	5.12550432	-39.50%
59.15015902	-0.01234251	-0.01704358	3.12349037	5.25849256	-40.60%
59.80016077	-0.00780495	-0.01669977	3.145584	5.39663059	-41.71%
60.45016252	-0.0024242	-0.01627019	3.16707088	5.54026116	-42.84%
61.10016426	0.00385591	-0.01574977	3.18793865	5.68975774	-43.97%
61.75016601	0.01108299	-0.01513364	3.20817534	5.84552813	-45.12%

Y Angle (deg)	Tan Shift	Sag Shift	Real Height	Ref. Height	Distortion
62.40016776	0.01929427	-0.01441728	3.22776956	6.00801844	-46.28%
63.05016951	0.0285151	-0.01359659	3.24671068	6.17771774	-47.44%
63.70017125	0.03875764	-0.01266801	3.264989	6.35516337	-48.62%
64.350173	0.0500199	-0.01162856	3.28259597	6.54094706	-49.81%
65.00017475	0.06228527	-0.01047599	3.29952431	6.73572199	-51.01%

表2-1 镜头畸变映射表

以表2-1 中Y Angle为X轴，Real Height为Y轴，其实际像高与入射角度的关系如图2-4所示，接近图2-3中的equidistant模型。

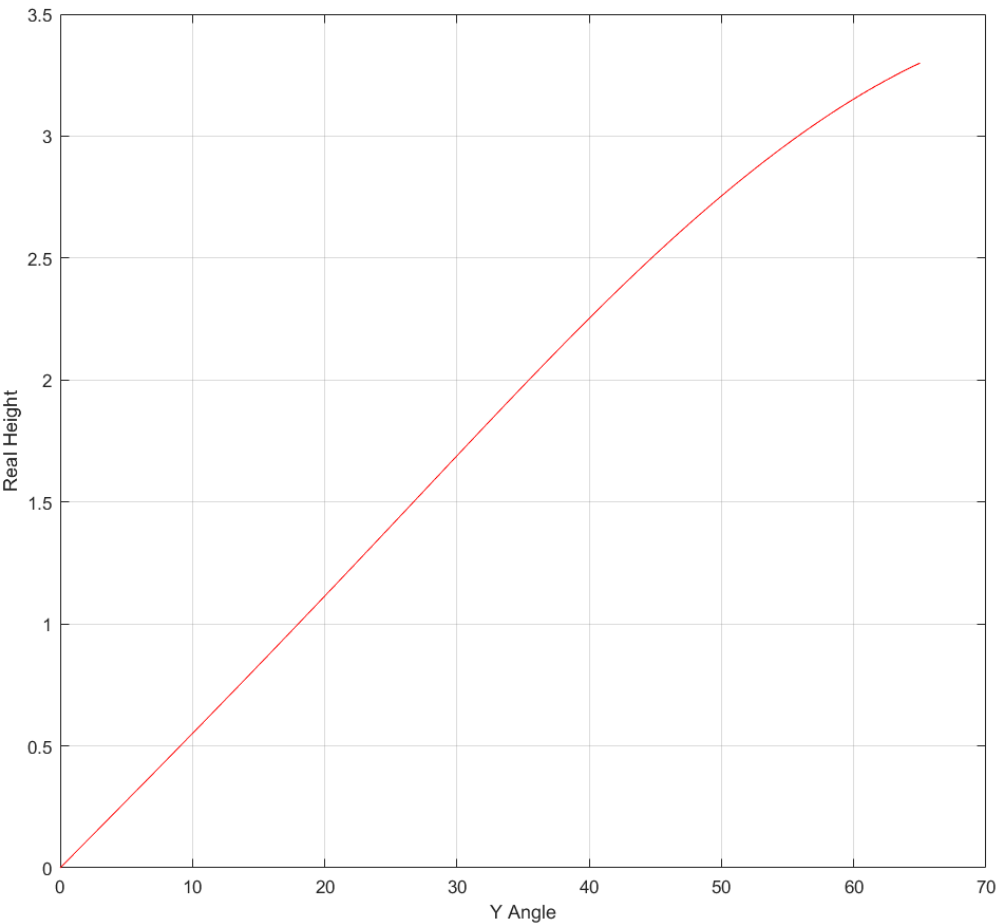


图2-4 实际像高与入射角的关系

图2-5为镜头畸变映射曲线，图（a）属于一般广角镜头（equidistant模型）的畸变映射曲线；图（b）是经过特殊畸变处理的镜头的畸变映射区域，绿色区域做了无畸变处理，导致畸变程度在某个fov下突然增大，畸变映射曲线呈现分段函数的特性。

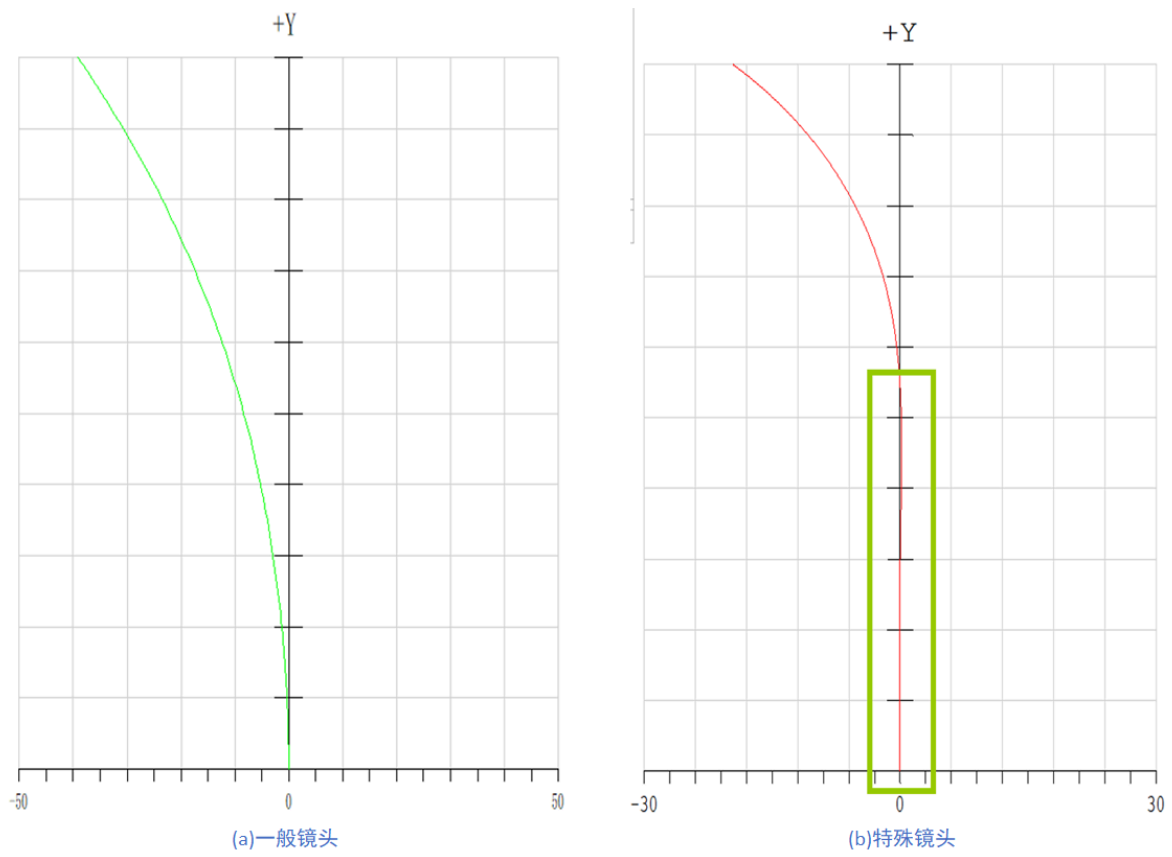


图2-5 畸变映射曲线图

以表2-1中Y Angle为Y轴，Distortion为X轴绘制畸变映射曲线，如图2-6所示。

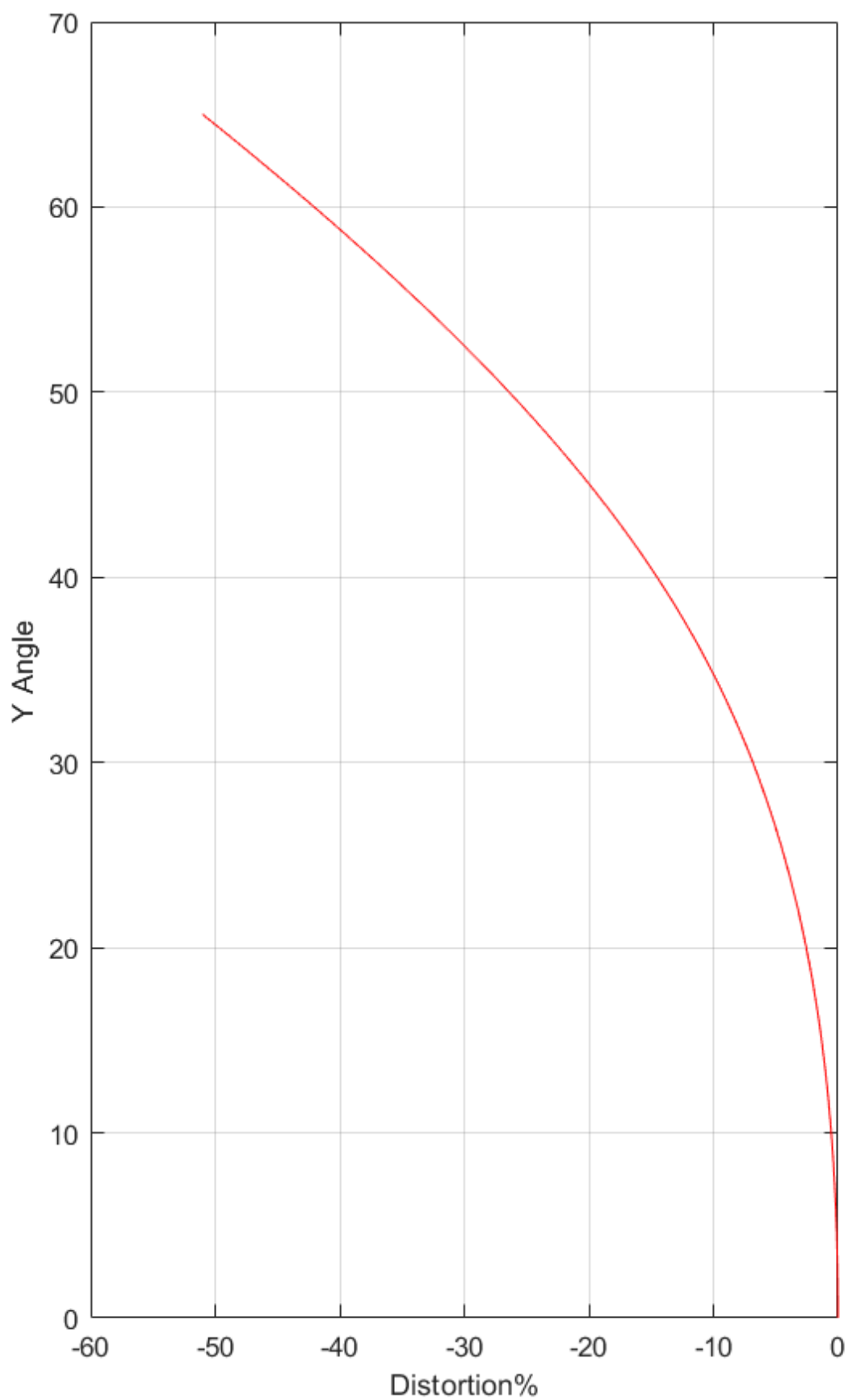


图2-6 表2-1数据畸变映射曲线

通过图2-6可知，其畸变映射曲线与图2-5中（a）相似，符合镜头选型要求。

2.2 模组设计实例

图2-7为RK双目模组实例图，其参数信息如下：

- sensor：GC2063
- 光轴水平夹角：60°、光轴垂直夹角：0°
- 单目水平FOV：109°、垂直FOV：59°
- 拼接水平FOV：170°、拼接垂直FOV：53°

本文的参数设计信息都是以该模组为例，包括棋盘格规格设计和标定场景布局。

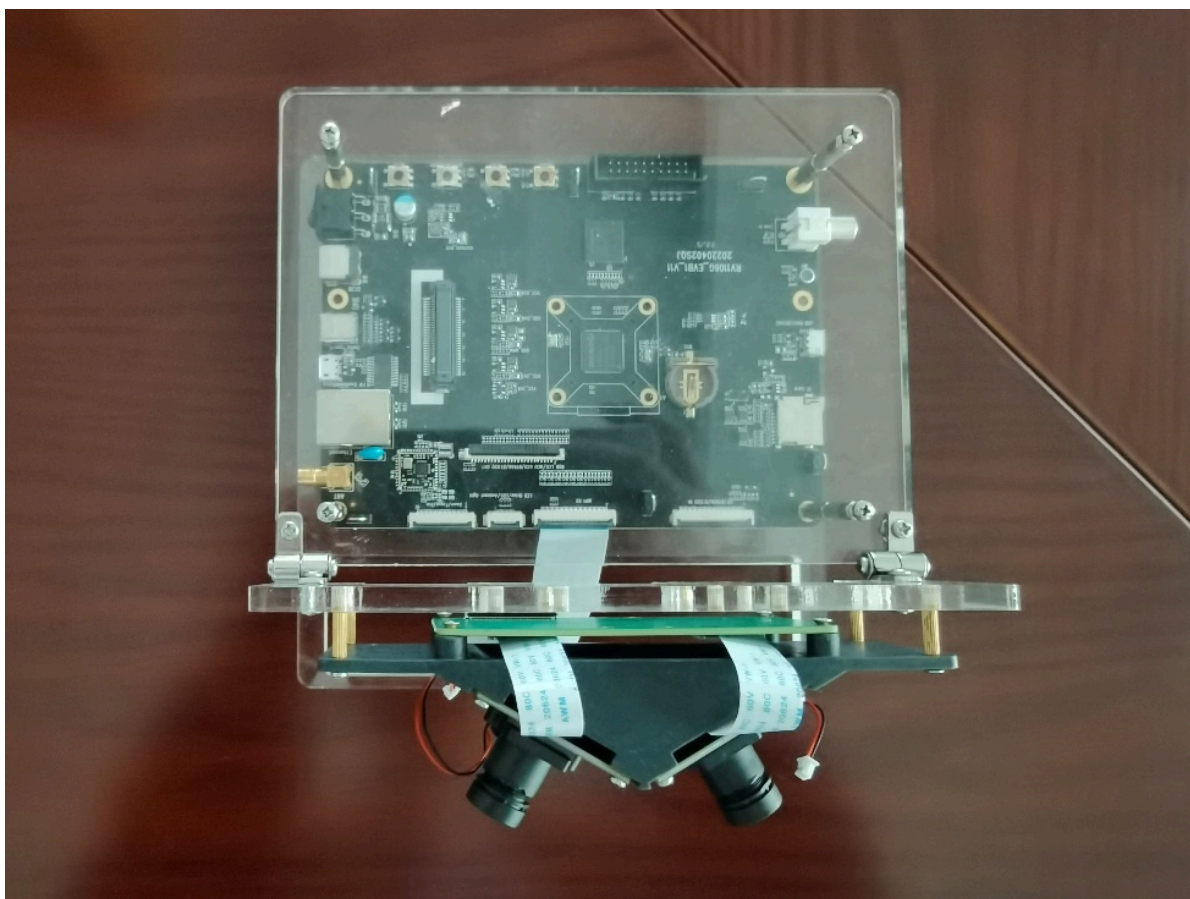


图2-7 RK双目模组实例

3. 双目产线标定

双目拼接模组产线标定通过对所提供棋盘格模具的单张拍摄，来获得立体相机模组的所有标定参数。

3.1 操作环境

3.1.1 操作环境设计

如图3-1所示，为模组标定时场景示意图。每次拍摄标定板时，要求模组的**左侧相机能够完整拍摄到左侧的四块棋盘格**如图中蓝色框所示，**右侧相机能够完整拍摄到右侧的四块棋盘格**如图中绿色框所示。

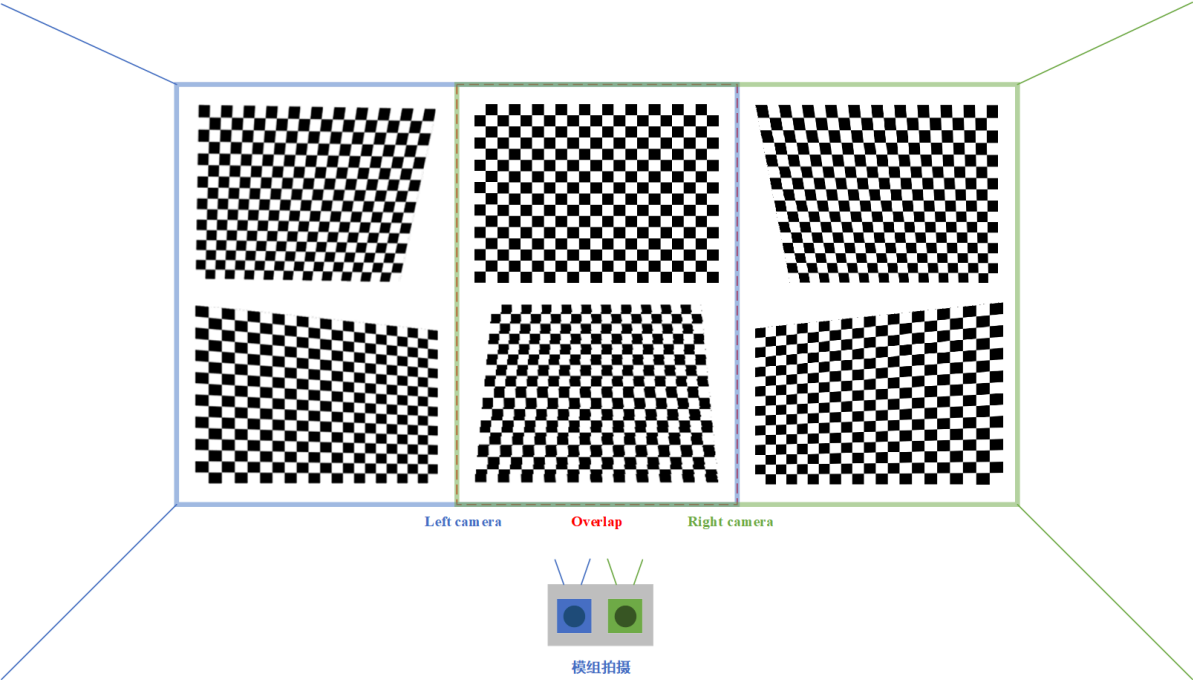


图3-1 操作环境

图3-2为六个棋盘格没有旋转时的摆放位置和棋盘格尺寸。其中外圈的红线表示两相机视野的边界，**标定板与标定板之间**有大于30mm的距离（可根据实际情况适当调整），每个棋盘格的尺寸为500mm*280mm（该尺寸以2-2中模组为例进行设计）。

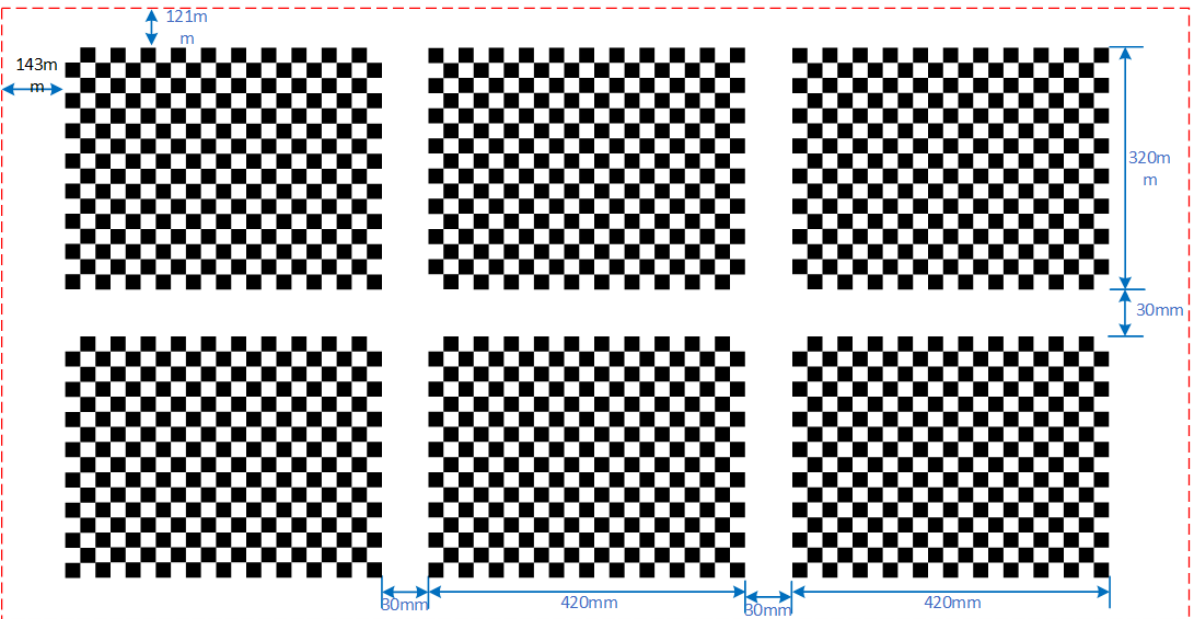


图3-2 棋盘格摆放位置（未旋转）

图3-3为正式标定时六个棋盘格的摆放位置。标定过程中需要注意：

- 任意一个相机所拍摄的**四个棋盘格不能在同一个平面上**，可根据棋盘格在视野中的位置进行适当调整每个棋盘格的旋转角度。

- 尽量保证6个棋盘格图像（包括中间留白）覆盖较大视野。
- 要求棋盘格必须全部处于两个相机的视野内，左侧四个棋盘格全部处于左侧相机的视野内，右侧四个棋盘格全部处于右侧相机的视野内（包括棋盘格留白区域）。

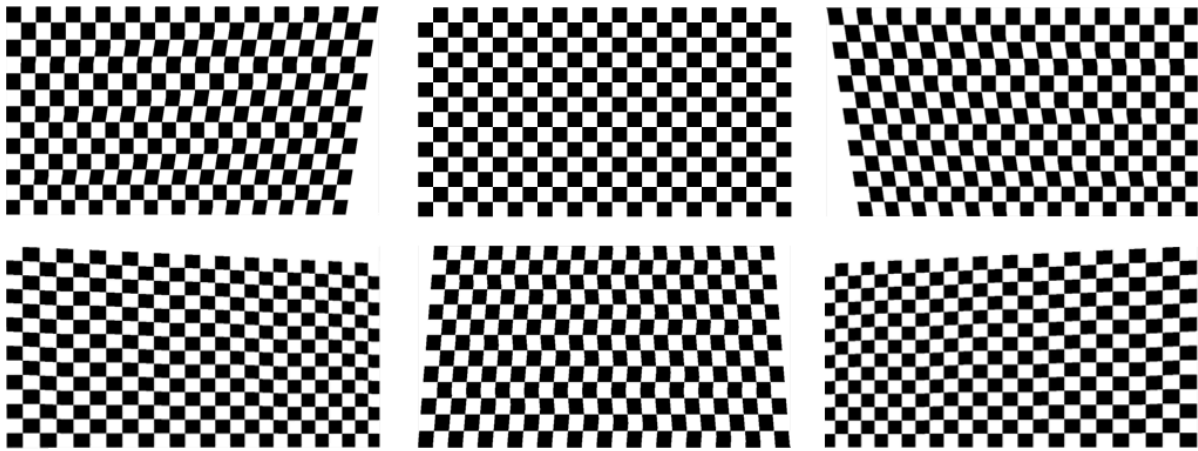


图3.3 棋盘格摆放位置（正式标定）

当两个相机重叠区域的夹角较小时，考虑将中间两个棋盘格旋转90度，如下图所示。

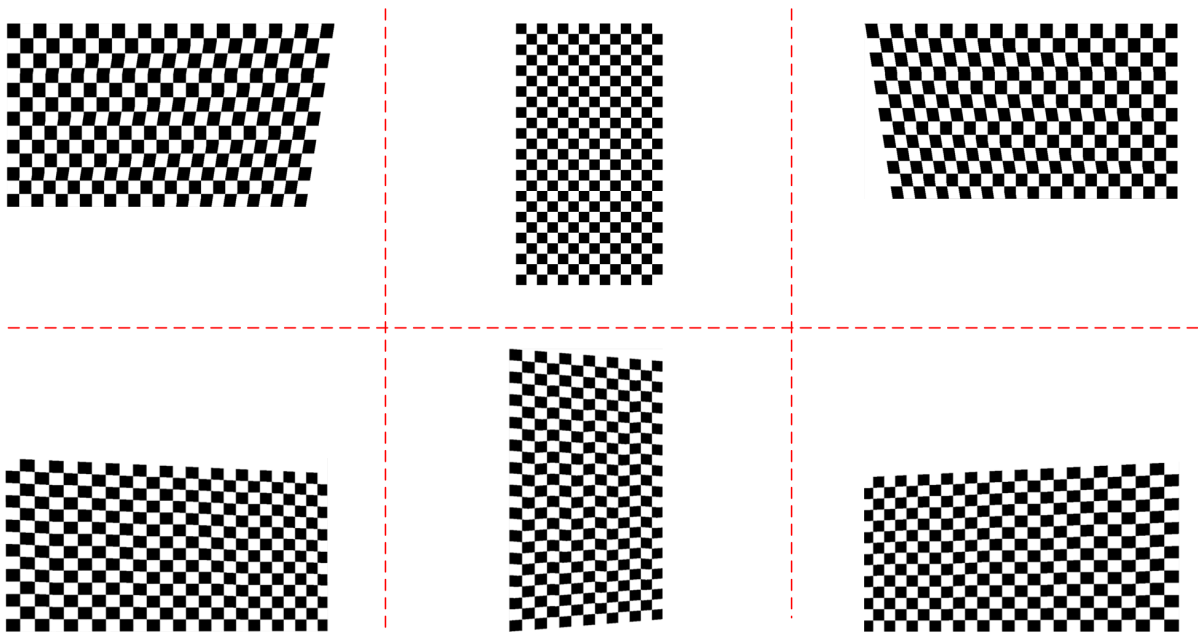


图3-4 棋盘格摆放位置（重叠区域夹角较小时）

3.1.2 操作环境实例

图3-5为操作环境实例，为了方便调整六个棋盘格所在位置，采用六个三脚架的结构。工厂产线标定时，可先确定拍摄距离和棋盘格摆设位置后根据实际需求设计成固定结构。如章节3.1.1中所述，棋盘格之间必须有一定间隔，六个棋盘格尽可能占据较大的相机视野。实际拍摄的图像如图3-6。

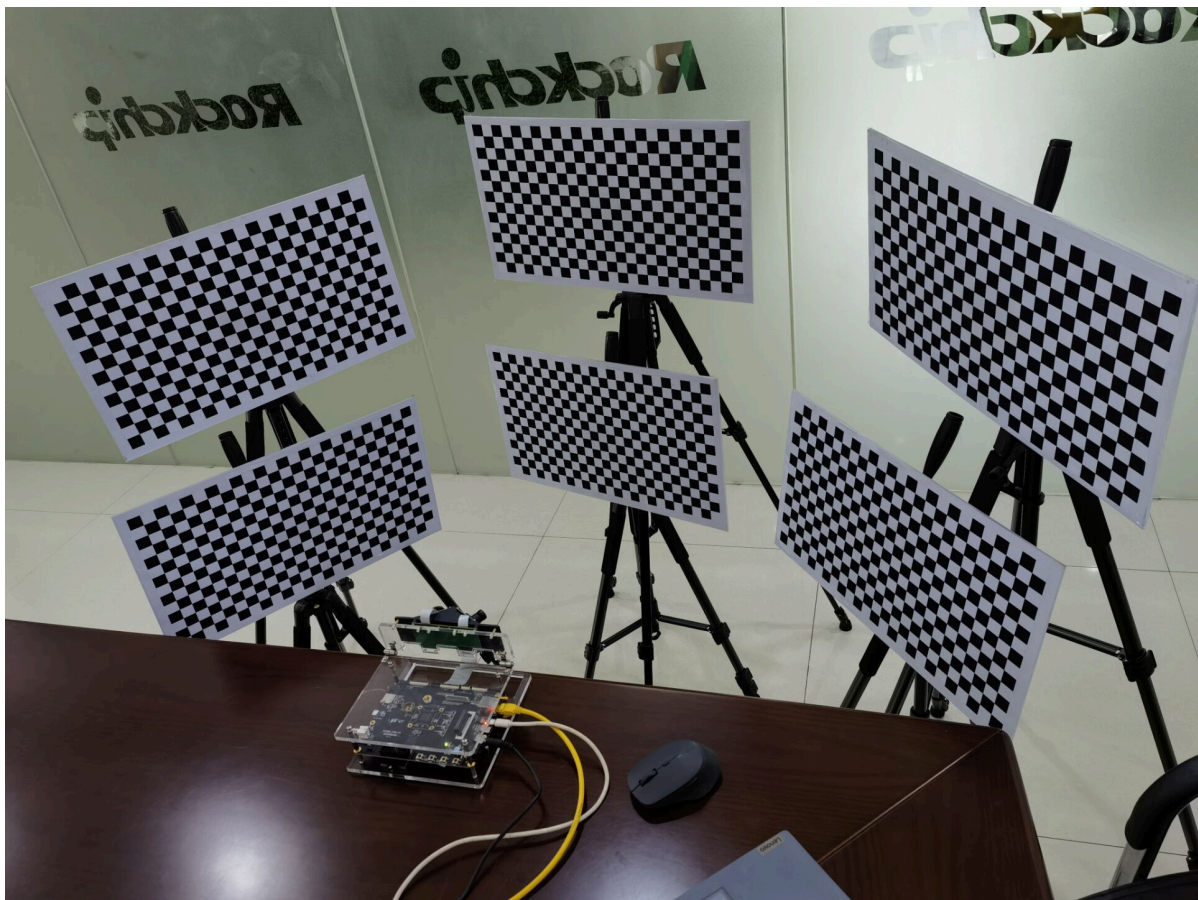


图3-5 操作环境实例

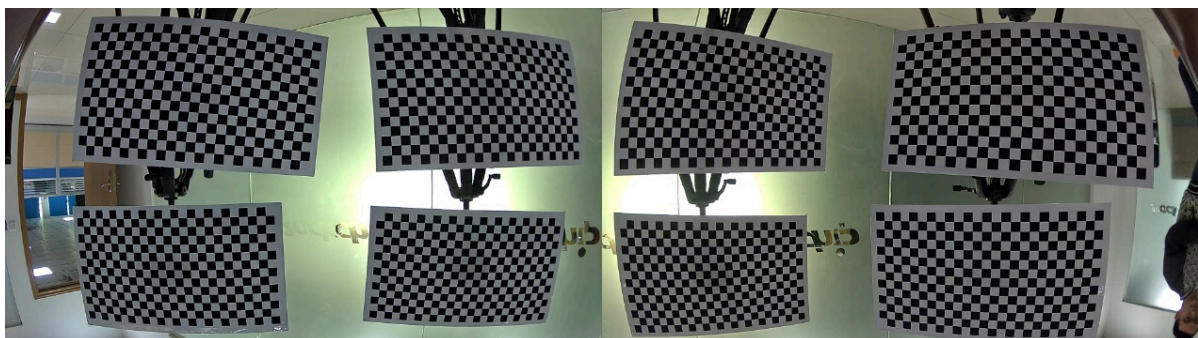


图3-6 拍摄预览图

3.2 标定场景布局定义

在标定场景中，6个不同棋盘格按照章节3.1所述进行布局。具体拍摄要求如下：

- 棋盘格距离定义为保证满足章节3.1所述覆盖面要求下的距离，同时在该距离下可以清晰的拍摄到棋盘格上的角点。
- 拍摄时，通过合理的相机曝光获取清晰的棋盘格图像。
- 要求棋盘格的长边与sensor的长边平行，**当重叠区域较小时，将中间两个棋盘格旋转90度摆放，即重叠区域的棋盘格短边与sensor的长边平行。**
- 拍摄的棋盘格图像中，黑白格灰度差应大于 80。理想状态下，白色格子的灰度值在 180 以上，黑色格子的灰度值在 100 以下。如果黑白格灰度无法始终满足该条件，则必须保证相邻黑白格的平均灰度差大于80。
- 环境光强度在500Lux-1000Lux范围内，尽量保证亮度均匀，棋盘格图像不出现过亮或者过暗的区域，同时角点清晰可见。

- 坐标系选择右手坐标系，X轴水平向右，Y轴竖直向下，Z轴垂直向里；
- 每个旋转都以过棋盘格中心的轴作为旋转轴。
- 棋盘格尽量占据较大FOV。
- 拍摄时背景最好是纯黑或者纯白背景色，背景纹理尽可能少（如背靠白墙拍摄）。
- 建议采用背光板拍摄，效果更佳。使用背光板时，其超出棋盘格的部分不能超过10mm。最好控制在0mm，即背光板与棋盘格边缘贴合。如果没有背光板，最好满足第9条要求。
- **FOV不同，模组到棋盘格的距离也应按需调整：**FOV变大，两者距离适当缩小；FOV变小则相反；对于该模组，两个相机需同时拍摄到两块居中的棋盘格，共同视野的FOV相对小，因此两个棋盘格适当放远，两边的四块棋盘格适当靠中。
- 建议将模组到棋盘格的距离可调节。

3.3 棋盘格规格设计

棋盘格规格设计要遵循以下条件：

- 指定标定距离下每个相机应拍摄到四张完整棋盘格；
- 四张棋盘格尽量覆盖相机视野的80%—90%。

以RK双目拼接模组为例，sensor和镜头信息如下：

- sensor：GC2063，像元尺寸 $2.8\mu\text{m} \times 2.8\mu\text{m}$ ，分辨率 1920×1080
- 焦距：2.8mm
- 拍摄距离：600mm
- 镜头水平FOV 109° ，镜头垂直FOV 59°

通过以上信息可以计算出拍摄距离为600mm处相机视野的尺寸：

- 水平尺寸： $1920 \times 2.8\mu\text{m} \times 600\text{mm} / 2.8\text{mm} = 1152\text{mm}$ ；
- 垂直尺寸： $1080 \times 2.8\mu\text{m} \times 600\text{mm} / 2.8\text{mm} = 648\text{mm}$ 。

假设棋盘格占据视野的90%，在600mm处拍摄时，棋盘格需要覆盖 $1037\text{mm} \times 584\text{mm}$ 的视野；由于每个相机视野需要四块棋盘格覆盖，同时棋盘格之间至少有30mm的距离，所以每个棋盘格尺寸按 $504\text{mm} \times 292\text{mm}$ 计算；假设每个格子尺寸为 $20\text{mm} \times 20\text{mm}$ ，那么最终棋盘格个数为 25×14 ，最终棋盘格尺寸为 500×280 。

在棋盘格设计过程中棋盘格图像中格子个数和棋盘格图像中每个格子的尺寸可根据实际需求进行调整，但棋盘格图像的边界处应保留大于15mm的留白区域。

根据上述计算模组标定中使用的棋盘格如图3-7所示，是对称的方形棋盘格，棋盘格的具体参数如下：

- 棋盘格的长为500mm，宽为280mm；
- 棋盘格中每个方块的物理尺寸是20mm，精度要求为0.1mm；
- 棋盘格要求表面平整；
- 可采用背投发光方式设计棋盘格；
- 棋盘格四周要有留白区域，最好存在大于15mm的留白区域；
- 棋盘格的长边共25个方格，短边共14个方格。

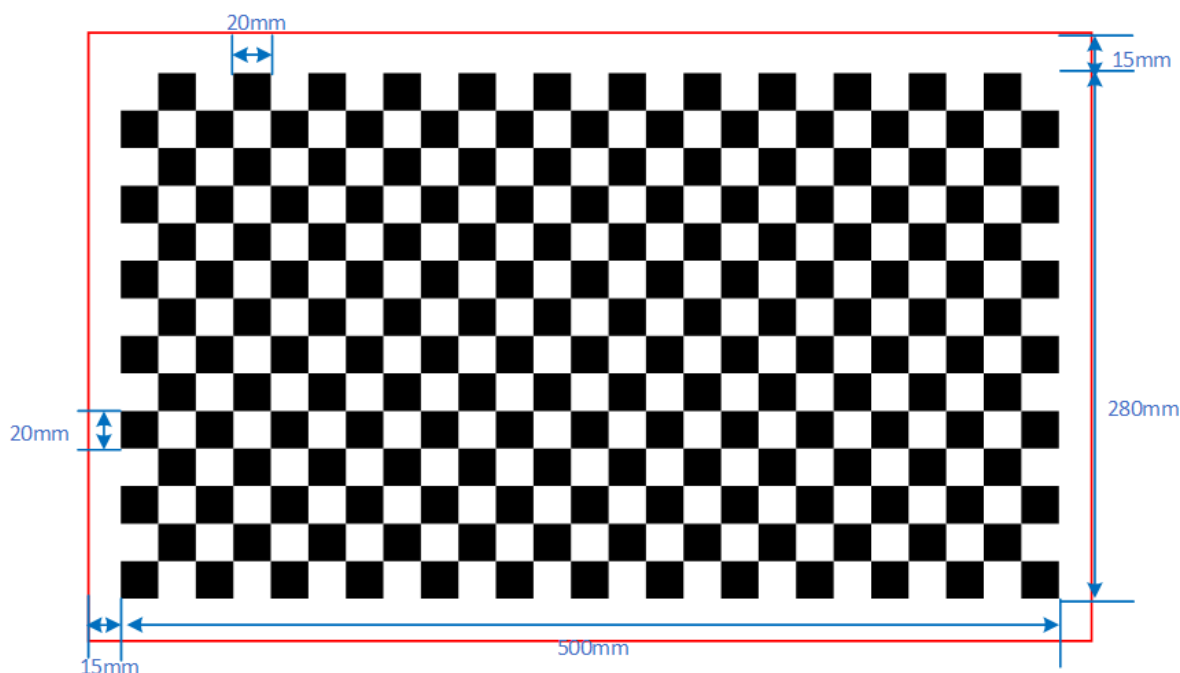


图3-7 棋盘格规格示意图

3.4 操作步骤

- 按照规格打印棋盘格，根据章节3.1描述摆放6个棋盘格；
- 将双摄像头放置在距离棋盘格合适的位置（保证拍摄视野满足设计需求），两个摄像头连线的中心对准6个棋盘格模具的中心，使左右摄像头均能拍摄到4个棋盘格；
- 双摄像头同步对焦至拍摄距离处，使得棋盘格显示清晰，同时拍摄获取左右图片；
- 将左右图及配置文件作为输入，获得立体标定的所有参数。

3.5 API接口及Demo

3.5.1 接口定义

```

/*
 * @enum    rkAVS_CALIB_STATUS_E
 * @brief    the return status of function
 */
typedef enum rkAVS_CALIB_STATUS_E
{
    RK_AVS_CALIB_STATUS_EOF = -1,           /**< error of function inside */
    RK_AVS_CALIB_STATUS_OK = 0,             /**< run successfully */
    RK_AVS_CALIB_STATUS_FILE_READ_ERROR,    /**< error: fail to read file */
    RK_AVS_CALIB_STATUS_FILE_WRITE_ERROR,   /**< error: fail to write file */
    RK_AVS_CALIB_STATUS_INVALID_PARAM,      /**< error: invalid parameter */
    RK_AVS_CALIB_STATUS_ALLOC_FAILED,       /**< error: fail to alloc buffer */
    RK_AVS_CALIB_STATUS_BUTT                /**< reserved fields */
}AVS_CALIB_STATUS_E;

```

rkAVS_CALIB_STATUS_E定义算法库内部的状态信息。

```

/*
 * @enum    rkAVS_CALIB_IMAGE_FORMAT_E
 * @brief    Specify the image format.
 * @note     Have gray, rgb, yuvnv12.
 */
typedef enum rkAVS_CALIB_IMAGE_FORMAT_E {
    RK_AVS_IMAGE_FORMAT_TYPE_GRAY = 0,           /**<gray format*/
    RK_AVS_IMAGE_FORMAT_TYPE_RGB = 1,            /**<rgb format*/
    RK_AVS_IMAGE_FORMAT_TYPE_YUVNV12 = 2,        /**<yuvnv12
format*/
} AVS_CALIB_IMAGE_FORMAT_E;

```

rkAVS_CALIB_IMAGE_FORMAT_E定义输入图像的基本格式，包括灰度图、RGB图和YUVNV12图。

```

/*
 * @enum    rkAVS_CALIB_IMAGE_PARAMS_S
 * @brief    calibration image params.
 */
typedef struct rkAVS_CALIB_IMAGE_PARAMS_S
{
    uint32_t u32ImageWidth;                       /**<image
width*/
    uint32_t u32ImageHeight;                       /**<image
height*/
    uint32_t u32ImageStride;                       /**<image
stride*/
    AVS_CALIB_IMAGE_FORMAT_E eImageFormat;         /**<image format*/
}AVS_CALIB_IMAGE_PARAMS_S

```

rkAVS_CALIB_IMAGE_PARAMS_S定义输入图像的基本信息，保留图像的宽、高、stride和图像格式。

```

/*
 * @enum    rkAVS_CALIB_IMAGE_PARAMS_S
 * @brief    calibration image params.
 */
typedef struct rkAVS_CALIB_IMAGE_DATA_S
{
    AVS_CALIB_IMAGE_PARAMS_S stImageParams;
    uint8_t* pu8ImageData;
}AVS_CALIB_IMAGE_DATA_S;

```

rkAVS_CALIB_IMAGE_DATA_S 定义输入图像的参数信息。

```

/*
 * @enum    rkAVS_CALIB_BOARD_PARAMS_S
 * @brief    chess board params.
 */
typedef struct rkAVS_CALIB_BOARD_PARAMS_S
{
    uint32_t u32BoardWidth;
    /**<coners number of board width*/
    uint32_t u32BoardHeight;
    <coners number of board height*/
    uint32_t u32SquareSize;
    /**<square size*/
}AVS_CALIB_BOARD_PARAMS_S;

```

rkAVS_CALIB_BOARD_PARAMS_S定义棋盘格的基本信息，包括棋盘格水平和垂直方向角点的个数u32BoardWidth、u32BoardHeight和棋盘格每个角点的尺寸(mm)。

```

/*
 * @enum    rkAVS_CALIB_CAMERA_TYPE_E
 * @brief    Specify the camera type.
 * @note    Have pin hole, CMei, fisheye.
 */
typedef enum rkAVS_CALIB_CAMERA_TYPE_E {
    AVS_CAMERA_TYPE_PINHOLE = 0,
    PIN HOLE*/
    AVS_CAMERA_TYPE_OMN = 1,
    CMei*/
    AVS_CAMERA_TYPE_FISHEYE = 2,
    <fisheye*/
} AVS_CALIB_CAMERA_TYPE_E;

```

rkAVS_CALIB_CAMERA_TYPE_E定义相机标定的模型。

```

/*
 * @enum    rkAVS_CALIB_CAMERA_PARAMS_S
 * @brief    camera params.
 */
typedef struct rkAVS_CALIB_CAMERA_PARAMS_S
{
    uint32_t u32CameraNum;
    <camera numbers*/
    uint32_t u32CameraFov;
    <camea fov*/
    AVS_CALIB_CAMERA_TYPE_E eCameraType;
    <camera type*/
}AVS_CALIB_CAMERA_PARAMS_S;

```

rkAVS_CALIB_MODEL_CAMERA_PARAMS_S定义相机的基本参数信息。

```

/*
 * @enum    AVS_STEREO_CALIB_CONFIG_S
 * @brief    input params.
 */

```

```

typedef struct AVS_STEREO_CALIB_CONFIG_S
{
    AVS_CALIB_IMAGE_DATA_S stImageData[MAX_CAMERA_NUM];           /**
<image params*/
    AVS_CALIB_BOARD_PARAMS_S stBoardParams;                       /**
<board params*/
    AVS_CALIB_CAMERA_PARAMS_S stCameraParams;                     /**
<camera params*/
    uint32_t u32IsSaveCornersImage;                               /**
<is save corners image*/
    uint32_t u32UseAccurateMode;                                  /**
<optimize by stitch*/
    const char* pcTorrentPath;                                    /**
<model calibration result path*/
    const char* pcCalibResultPath;                                /**
<calibration result path*/
}AVS_STEREO_CALIB_CONFIG_S;

```

AVS_STEREO_CALIB_CONFIG_S为双目产线标定的输入参数，包括图像信息、棋盘格信息、相机信息、是否保存角点的Flag、是否采用精确模式的flag、模型标定文件保存路径以及最终标定结果保存路径。

```

/*
 * @enum    rkAVS_STEREO_CALIB_POSE_S
 * @brief   output params.
 */
typedef struct rkAVS_STEREO_CALIB_POSE_S
{
    float f32RotationAngle[3]
    float f32Translation[3];
}AVS_STEREO_CALIB_POSE_S;

```

rkAVS_STEREO_CALIB_POSE_S定义模组的位姿信息，包括旋转角度和位移量。

```

/*
 * @enum    rkAVS_STEREO_CALIB_RMS_S
 * @brief   output params.
 */
typedef struct rkAVS_STEREO_CALIB_RMS_S
{
    float f32LeftInternalRms;
    float f32RightInternalRms;
    float f32StereoExternalRms
}AVS_STEREO_CALIB_RMS_S;

```

rkAVS_STEREO_CALIB_RMS_S定义产线标定的误差，包括内参标定误差和外参标定误差，**该误差值越小越好，大于0.5效果较差。**

```

/*
 * @enum    rkAVS_STEREO_CALIB_CHECK_S
 * @brief   output params.
 */
typedef struct rkAVS_STEREO_CALIB_CHECK_S
{
    AVS_STEREO_CALIB_POSE_S stPose;
    AVS_STEREO_CALIB_RMS_S stRms;
}AVS_STEREO_CALIB_CHECK_S;

```

rkAVS_STEREO_CALIB_CHECK_S为双目产线标定的输出参数，用于判别标定效果的好坏。

```
int32_t rkAVS_getCalibVersion(char avsToolVersion[128]);
```

rkAVS_getCalibVersion用于获取算法库的版本信息。

```
int32_t rkAVS_calibProductStereo(AVS_STEREO_CALIB_CONFIG_S* psInputParams,
    AVS_STEREO_CALIB_CHECK_S* stStereoCheckParams);
```

rkAVS_calibProductStereo用于双目模组产线标定。

3.5.2 标定Demo

```

leftImagePath: ./calibProduct/src/left.yuv
rightImagePath: ./calibProduct/src/right.yuv
dstPath: ./calibProduct/rk_calib_result/
calibTorrentPath: ./calibProduct/src/rk_2_camera_result.xml
cameraNum: 2 //camera number (2)
cameraType: 1 //0-pinhole、1-omn、
2-fisheye
cameraFov: 120 //horizontal fov
boardWidth: 15 //number of
chessboard corners in horizontal direction
boardHeight: 8 //number of
chessboard corners in vertical direction
squareSize: 30 //size of
chessboard(mm)
leftImageWidth: 1920 // camera0 image
width
leftImageHeight: 1080 // camera0 image
height
leftImageStride: 1920 // camera0 image
stride
rightImageWidth: 1920 // camera1 image
width
rightImageHeight: 1080 // camera1 image
height
rightImageStride: 1920 // camera1 image
stride
imageFormat: 2 //0-gray、1-rgb、2-
yuv

```

```

isSaveCornersImage: 1 //draw corners and
save
useAccurateMode: 1 //optimize by
stitch

```

以上为产线标定时算法库的参考配置参数。

```

//保存产线标定筛选参数
int saveCheckParams(std::string saveThreshPath, AVS_STEREO_CALIB_CHECK_S*
stereoCalibCheck)
{
    std::ofstream fout(saveThreshPath);
    if (!fout)
    {
        std::cout << "Open save_thresh_path fail!" << std::endl;
        return 0;
    }
    else
    {
        fout << "leftInternalRms: " << stereoCalibCheck->stRms.f32LeftInternalRms
<< std::endl;
        fout << "rightInternalRms: " << stereoCalibCheck-
>stRms.f32RightInternalRms << std::endl;
        fout << "externalRms: " << stereoCalibCheck->stRms.f32StereoExternalRms
<< std::endl;
        fout << "euler_angles_x: " << stereoCalibCheck-
>stPose.f32RotationAngle[0] << std::endl;
        fout << "euler_angles_y: " << stereoCalibCheck-
>stPose.f32RotationAngle[1] << std::endl;
        fout << "euler_angles_z: " << stereoCalibCheck-
>stPose.f32RotationAngle[2] << std::endl;
        fout << "t_x: " << stereoCalibCheck->stPose.f32Translation[0] <<
std::endl;
        fout << "t_y: " << stereoCalibCheck->stPose.f32Translation[1] <<
std::endl;
        fout << "t_z: " << stereoCalibCheck->stPose.f32Translation[2] <<
std::endl;
        //fout << "is_calib_succeed: " << is_succ << std::endl;
    }
    fout.close();
    return 1;
}

```

saveCheckParams用于保存产线标定的误差及模组信息，该参数可作为标定是否符合预期的判别参数。

```

bool readBinParams(std::string image_path, char* output_params, uint64_t length)
{
    std::ifstream fin;
    //读取raw图
    fin.open(image_path, std::ios::binary);
    if (!fin) {
        std::cerr << "open failed: " << image_path << std::endl;
        return false;
    }
}

```

```

// read函数读取（拷贝）流中的length各字节到buffer
fin.read((char*)output_params, length);
fin.close();
return true;
}

```

readBinParams用于读取YUV数据。

```

//产线标定读取配置文件
int readCalibProdictConfig(std::string readPath, std::vector<std::string>&
stringData, std::vector<int32_t>& int32Data)
{
    std::string stringLine;
    std::ifstream ifs(readPath, std::ifstream::in);
    if (ifs.fail())
    {
        std::cout << "read calib product config fail!" << std::endl;
    }
    int32_t lineNum = 0;
    int32_t valueData = 0;
    while (getline(ifs, stringLine))
    {
        std::stringstream ss(stringLine);
        std::string stringUnit;
        std::getline(ss, stringUnit, ' ');
        std::getline(ss, stringUnit, ' ');
        if (lineNum < 4)
        {
            stringData.push_back(stringUnit);
        }
        else
        {
            valueData = atoi(stringUnit.c_str());
            int32Data.push_back(valueData);
        }
        lineNum++;
    }
    ifs.close();
    return 0;
}

```

readCalibProdictConfig用于读取产线标定的配置信息。

```

//产线标定Demo
int stereoCalibProdDemo(std::string configPath)
{
    std::vector<std::string> stringData;
    std::vector<int32_t> int32Data;
    readCalibProdictConfig(configPath, stringData, int32Data);
    //获取版本信息
    char avsCalibVersion[128];
    rkAVS_getCalibVersion(avsCalibVersion);

    //初始化参数

```

```

AVS_STEREO_CALIB_CONFIG_S stInputParams;
//相机规格
stInputParams.stCameraParams.u32CameraNum = int32Data[0];
stInputParams.stCameraParams.u32CameraFov = int32Data[2];
switch (int32Data[1])
{
case 0:
    stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_PINHOLE;
    break;
case 1:
    stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_OMN;
    break;
case 2:
    stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_FISHEYE;
    break;
default:
    std::cout << "RK_AVS_CALIB: CameraType error!" << std::endl;
    return 1;
}
//棋盘格规格
stInputParams.stBoardParams.u32BoardWidth = int32Data[3];
stInputParams.stBoardParams.u32BoardHeight = int32Data[4];
stInputParams.stBoardParams.u32SquareSize = int32Data[5];
//图像读取路径
std::string leftImagePath = stringData[0];
std::string rightImagePath = stringData[1];
//图像规格
uint8_t* leftImage;
uint8_t* rightImage;
cv::Mat leftImageMat, rightImageMat;
int32_t leftDataSize, rightDataSize;
for (int32_t cam = 0; cam < stInputParams.stCameraParams.u32CameraNum; cam++)
{
    stInputParams.stImageData[cam].stImageParams.u32ImageWidth = int32Data[6
+ cam * 3];
    stInputParams.stImageData[cam].stImageParams.u32ImageHeight = int32Data[7
- cam * 3];
    stInputParams.stImageData[cam].stImageParams.u32ImageStride = int32Data[8
+ cam * 3];
}
switch (int32Data[12])
{
case 0:
    stInputParams.stImageData[0].stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_GRAY;
    stInputParams.stImageData[1].stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_GRAY;
    leftImageMat = cv::imread(leftImagePath, 0);
    rightImageMat = cv::imread(rightImagePath, 0);
    leftImage = leftImageMat.ptr<uint8_t>(0);
    rightImage = rightImageMat.ptr<uint8_t>(0);
    break;
case 1:
    stInputParams.stImageData[0].stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_RGB;

```

```

        stInputParams.stImageData[1].stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_RGB;
        leftImageMat = cv::imread(leftImagePath, 1);
        rightImageMat = cv::imread(rightImagePath, 1);
        leftImage = leftImageMat.ptr<uint8_t>(0);
        rightImage = rightImageMat.ptr<uint8_t>(0);
        break;
    case 2:
        stInputParams.stImageData[0].stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_YUVNV12;
        stInputParams.stImageData[1].stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_YUVNV12;
        leftDataSize = stInputParams.stImageData[0].stImageParams.u32ImageStride
* stInputParams.stImageData[0].stImageParams.u32ImageHeight * 3 / 2;
        rightDataSize = stInputParams.stImageData[1].stImageParams.u32ImageStride
* stInputParams.stImageData[1].stImageParams.u32ImageHeight * 3 / 2;
        leftImageMat =
cv::Mat(stInputParams.stImageData[0].stImageParams.u32ImageHeight * 3 / 2,
stInputParams.stImageData[0].stImageParams.u32ImageStride, CV_8UC1);
        rightImageMat =
cv::Mat(stInputParams.stImageData[1].stImageParams.u32ImageHeight * 3 / 2,
stInputParams.stImageData[1].stImageParams.u32ImageStride, CV_8UC1);
        leftImage = leftImageMat.ptr<uint8_t>(0);
        rightImage = rightImageMat.ptr<uint8_t>(0);
        readBinParams(leftImagePath, (char*)leftImage, leftDataSize);
        readBinParams(rightImagePath, (char*)rightImage, rightDataSize);
        break;
    default:
        std::cout << "RK_AVS_CALIB: ImageFormate error!" << std::endl;
        return 1;
}

//功能开关
stInputParams.u32IsSaveCornersImage = int32Data[13];
stInputParams.u32UseAccurateMode = int32Data[14];
//输入种子文件路径
stInputParams.pcTorrentPath = stringData[3].c_str();
//输入输出路径
std::string calibResultPath = stringData[2] + "rk_2_camera_result.xml";
stInputParams.pcCalibResultPath = calibResultPath.c_str();

//读取图像
stInputParams.stImageData[0].pu8ImageData = leftImage;
stInputParams.stImageData[1].pu8ImageData = rightImage;

//计时函数
#ifdef _WIN32
    clock_t startTime, endTime, timeAll;
    startTime = clock();
#else
    struct timeval startTime, endTime; // 秒+微妙
    time_t timeAll, timeAll_us;
    gettimeofday(&startTime, NULL);
#endif
//*****产线拼接
//*****

```

```

    AVS_STEREO_CALIB_CHECK_S stStereoCheckParams;
    int32_t status = rkAVS_calibProductStereo(&stInputParams,
&stStereoCheckParams);
    if (status)
    {
        printf("rkAVS_calibProductStereo fail!\n");
    }
    else
    {
        printf("rkAVS_calibProductStereo OK!\n");
    }
#ifdef _WIN32
    endTime = clock();
    timeAll = endTime - startTime;
    printf("rkAVS_calibProductStereo: %d ms\n", timeAll);
#else
    gettimeofday(&endTime, NULL);
    timeAll_us = 1000000 * (endTime.tv_sec - startTime.tv_sec) + (endTime.tv_usec
- startTime.tv_usec);
    printf("rkAVS_calibProductStereo: %d us\n", timeAll_us);
#endif

    //保存产线筛选参数
    std::string saveCheckPath = stringData[2] + "productCheckParams.txt";
    saveCheckParams(saveCheckPath, &stStereoCheckParams);
    return 0;
}

```

calibProdDemo为产线标定调用算法库完成标定的Demo。

4. 双目产线验证

模组验证的目的是提前筛选问题模组，提高出厂良率。双目产线验证分为客观评测和主观评测两种模式，客观评测通过拍摄棋盘格，利用标定文件计算棋盘格的重投影误差，输出误差值；主观评测是利用标定文件输出拼接图，人眼观测拼接效果。

4.1 客观评测

客观评测的验证过程主要利用模组标定参数即标定输出的xml文件，对指定验证标定板进行验证，利用检测结果与实际值进行对比，进而筛选出有问题的模组。

4.1.1 操作环境

如图4-1所示，每次拍摄标定板时，要求模组的**两个相机能够同时拍摄到两块完整的棋盘格**。

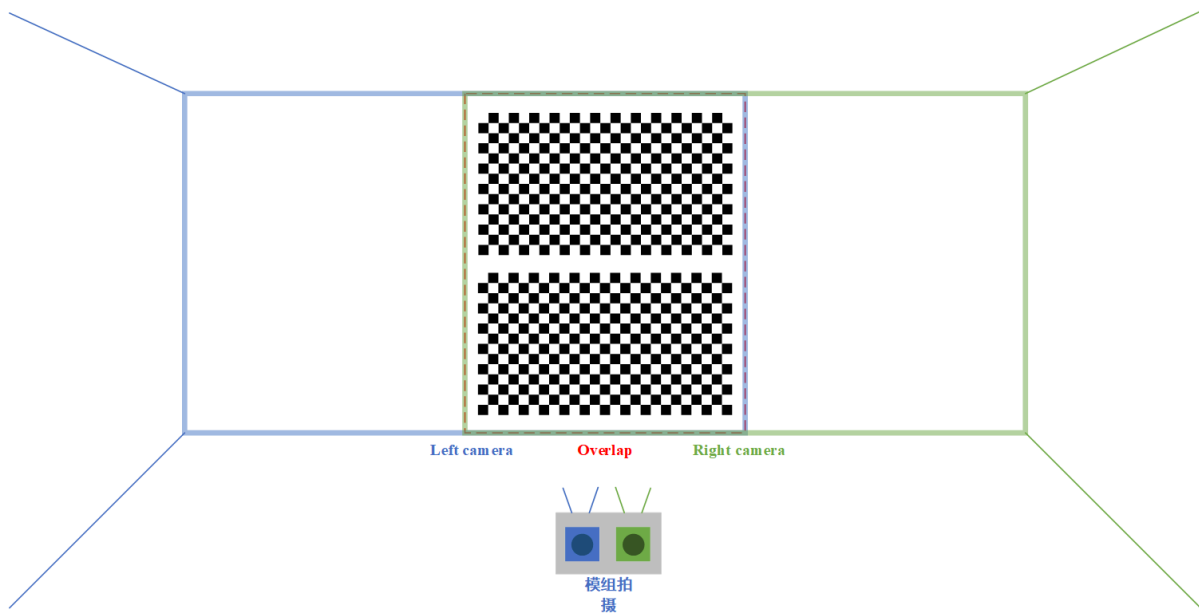


图4-1 验证环境示意图

4.1.2 棋盘格规格设计

每个棋盘格的规格与标定场景一致，具体可参考3.3。

4.1.3 API接口及Demo

4.1.3.1 接口定义

```
int32_t rkAVS_verifyProductStereo(AVS_STEREO_CALIB_CONFIG_S* psInputParams,
float* stereoRms);
```

rkAVS_verifyProductStereo用于验证标定参数的准确性，输出结果stereoRms越小越好，一般要求小于0.5。

4.1.3.2 验证Demo

```
//产线标定验证Demo
int stereoVerifyProdDemo(std::string configPath)
{
    std::vector<std::string> stringData;
    std::vector<int32_t> int32Data;
    readCalibProdictConfig(configPath, stringData, int32Data);
    //获取版本信息
    char avsCalibVersion[128];
    rkAVS_getCalibVersion(avsCalibVersion);

    //初始化参数
    AVS_STEREO_CALIB_CONFIG_S stInputParams;
    //相机规格
    stInputParams.stCameraParams.u32CameraNum = int32Data[0];
    stInputParams.stCameraParams.u32CameraFov = int32Data[2];
```

```

switch (int32Data[1])
{
case 0:
    stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_PINHOLE;
    break;
case 1:
    stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_OMN;
    break;
case 2:
    stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_FISHEYE;
    break;
default:
    std::cout << "RK_AVS_CALIB: CameraType error!" << std::endl;
    return 1;
}

//棋盘格规格
stInputParams.stBoardParams.u32BoardWidth = int32Data[3];
stInputParams.stBoardParams.u32BoardHeight = int32Data[4];
stInputParams.stBoardParams.u32SquareSize = int32Data[5];
//图像读取路径
std::string leftImagePath = stringData[0];
std::string rightImagePath = stringData[1];
//图像规格
uint8_t* leftImage;
uint8_t* rightImage;
cv::Mat leftImageMat, rightImageMat;
int32_t leftDataSize, rightDataSize;
for (int32_t cam = 0; cam < stInputParams.stCameraParams.u32CameraNum; cam++)
{
    stInputParams.stImageData[cam].stImageParams.u32ImageWidth = int32Data[6
+ cam * 3];
    stInputParams.stImageData[cam].stImageParams.u32ImageHeight = int32Data[7
+ cam * 3];
    stInputParams.stImageData[cam].stImageParams.u32ImageStride = int32Data[8
+ cam * 3];
}
switch (int32Data[12])
{
case 0:
    stInputParams.stImageData[0].stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_GRAY;
    stInputParams.stImageData[1].stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_GRAY;
    leftImageMat = cv::imread(leftImagePath, 0);
    rightImageMat = cv::imread(rightImagePath, 0);
    leftImage = leftImageMat.ptr<uint8_t>(0);
    rightImage = rightImageMat.ptr<uint8_t>(0);
    break;
case 1:
    stInputParams.stImageData[0].stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_RGB;
    stInputParams.stImageData[1].stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_RGB;
    leftImageMat = cv::imread(leftImagePath, 1);
    rightImageMat = cv::imread(rightImagePath, 1);
    leftImage = leftImageMat.ptr<uint8_t>(0);

```

```

        rightImage = rightImageMat.ptr<uint8_t>(0);
        break;
    case 2:
        stInputParams.stImageData[0].stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_YUVNV12;
        stInputParams.stImageData[1].stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_YUVNV12;
        leftDataSize = stInputParams.stImageData[0].stImageParams.u32ImageStride
* stInputParams.stImageData[0].stImageParams.u32ImageHeight * 3 / 2;
        rightDataSize = stInputParams.stImageData[1].stImageParams.u32ImageStride
* stInputParams.stImageData[1].stImageParams.u32ImageHeight * 3 / 2;
        leftImageMat =
cv::Mat(stInputParams.stImageData[0].stImageParams.u32ImageHeight * 3 / 2,
stInputParams.stImageData[0].stImageParams.u32ImageStride, CV_8UC1);
        rightImageMat =
cv::Mat(stInputParams.stImageData[1].stImageParams.u32ImageHeight * 3 / 2,
stInputParams.stImageData[1].stImageParams.u32ImageStride, CV_8UC1);
        leftImage = leftImageMat.ptr<uint8_t>(0);
        rightImage = rightImageMat.ptr<uint8_t>(0);
        readBinParams(leftImagePath, (char*)leftImage, leftDataSize);
        readBinParams(rightImagePath, (char*)rightImage, rightDataSize);

        break;
    default:
        std::cout << "RK_AVS_CALIB: ImageFormate error!" << std::endl;
        return 1;
}
//功能开关
stInputParams.u32IsSaveCornersImage = int32Data[13];
stInputParams.u32UseAccurateMode = int32Data[14];
//输入种子文件路径
stInputParams.pcTorrentPath = NULL;
//输入输出路径
std::string calibResultPath = stringData[2] + "rk_2_camera_result.xml";
//标定参数所在的输入路径
stInputParams.pcCalibResultPath = calibResultPath.c_str();

//读取图像
stInputParams.stImageData[0].pu8ImageData = leftImage;
stInputParams.stImageData[1].pu8ImageData = rightImage;
//模型标定

//计时函数
#ifdef _WIN32
    clock_t startTime, endTime, timeAll;
    startTime = clock();
#else
    struct timeval startTime, endTime; // 秒+微妙
    time_t timeAll, timeAll_us;
    gettimeofday(&startTime, NULL);
#endif
//*****产线拼接
//*****
float stVerifyRms;
int32_t status = rkAVS_verifyProductStereo(&stInputParams, &stVerifyRms);
if (status)

```

```

    {
        printf("rkAVS_calibProductStereo fail!\n");
    }
#ifdef _WIN32
    endTime = clock();
    timeAll = endTime - startTime;
    printf("rkAVS_verifyProductStereo: %d ms\n", timeAll);
#else
    gettimeofday(&endTime, NULL);
    timeAll_us = 1000000 * (endTime.tv_sec - startTime.tv_sec) + (endTime.tv_usec
- startTime.tv_usec);
    printf("rkAVS_verifyProductStereo: %d us\n", timeAll_us);
#endif
    printf("verify rms is: %f", stVerifyRms);
    return 0;
}

```

4.2 主观评测

主观评测通过将标定文件导入板端，运行AVS获取拼接输出图，通过人眼观测拼接图拼缝处的对齐效果进行筛选，主要步骤如下：

- 确认拼接效果验证的环境：光滑的墙面，大尺寸棋盘格（将棋盘格张贴在墙面上）
- 将标定文件导入板端
- 板端运行AVS，并设置最佳拼接距离为相机到墙面的距离
- 调整模组姿态，使得拼缝落在墙面张贴的棋盘格上
- 观测拼缝处的拼接效果：拼缝处没有显著错位认为标定效果满足需求，如果存在显著错位认为标定效果不佳需要重新标定

5. 多目产线标定

多目产线标定是针对两目及两目以上的拼接模组，其包含便捷模式和快速模式。文档以四目环形拼接模组为例，介绍多目产线标定的实现过程，下图为RK的四目拼接模组样机。

- 便捷模式：两两相机单独拍摄，参考双目拼接模组拍图
- 快速模式：多个相机同时拍图，拍摄一组图像即可

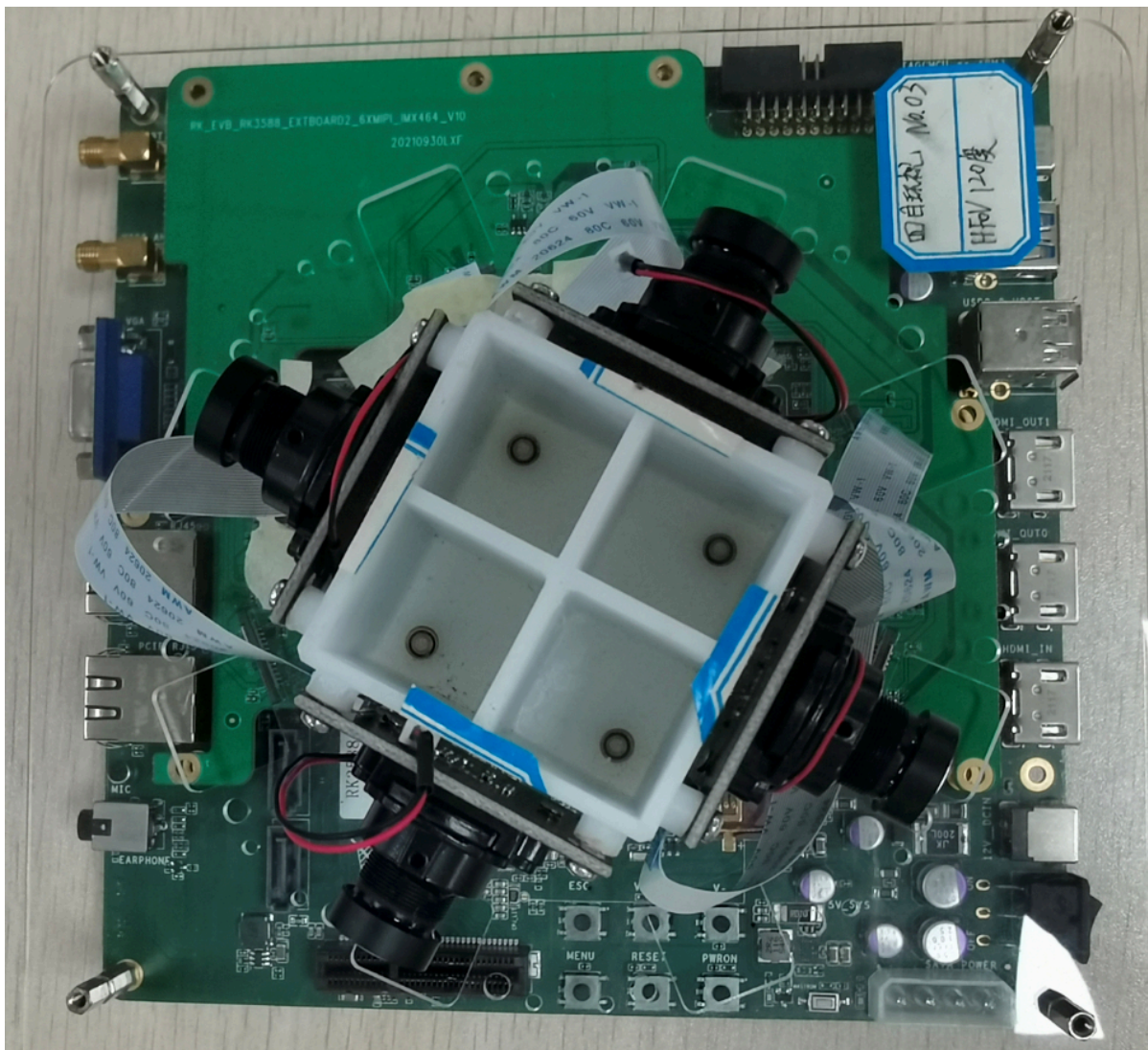


图5-1 RK四目环视样机

5.1 便捷模式

便捷模式针对产线标定场所有限的情况下，通过摆放较少的棋盘格通过多次拍摄达到产线标定的目的。其拍摄次数与模组两两相机间重叠区域的个数有关，一般多摄像模组有几个重叠带就会拍摄几组图像。

5.1.1 拍摄场景

该模式的操作环境及场景布局及棋盘格规格设计与章节3相同，如图5-2所示为实际拍摄场景。

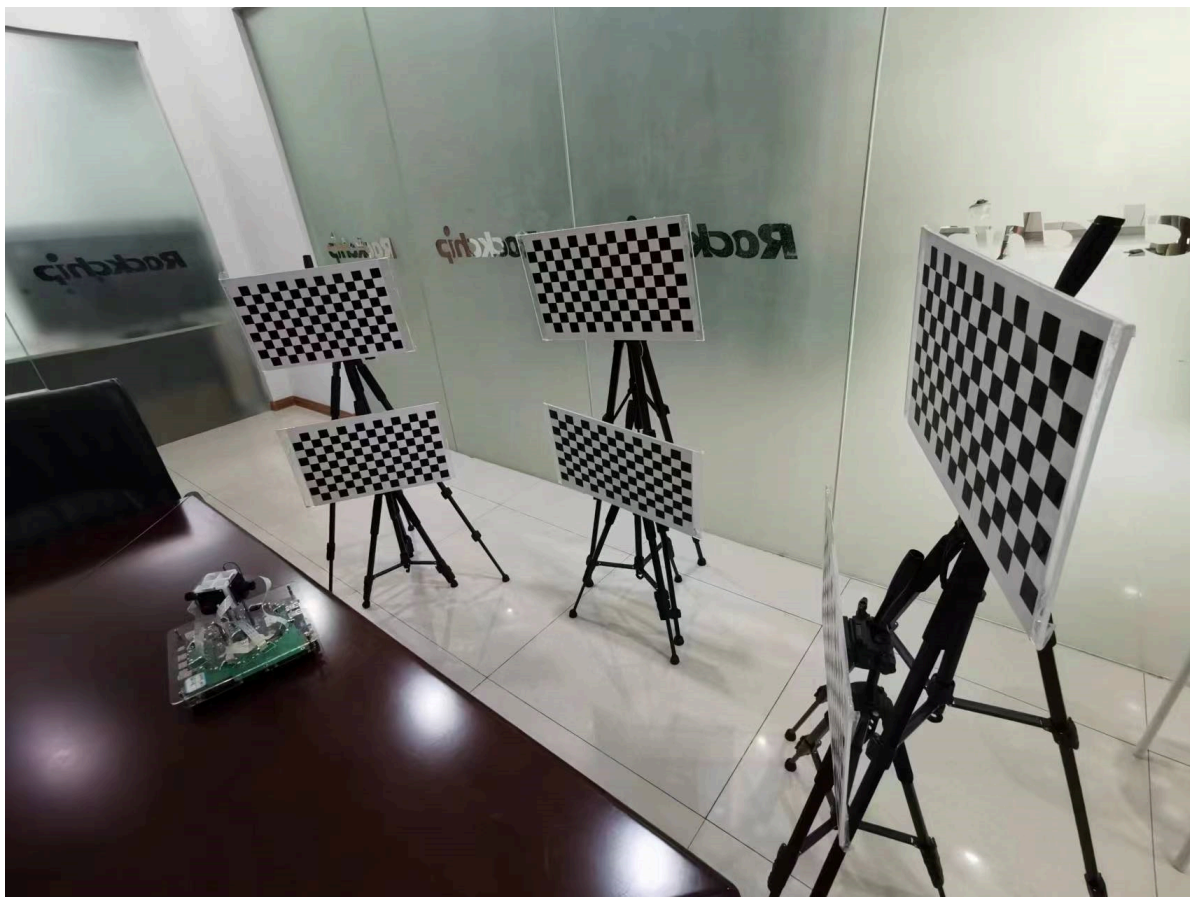


图5-2 便捷模式场景布局

5.1.2 拍图流程

其主要步骤如下：

- 根据3.2步骤完成标定环境准备
- 如图5-3所示先拍摄camera0和camera1的一组标定图
- 将模组转置90度，再拍摄camera1和camera2的一组标定图
- 依次得到camera2、camera3的标定图以及camera3、camera0的标定图
- 按照指定规则对图像命名

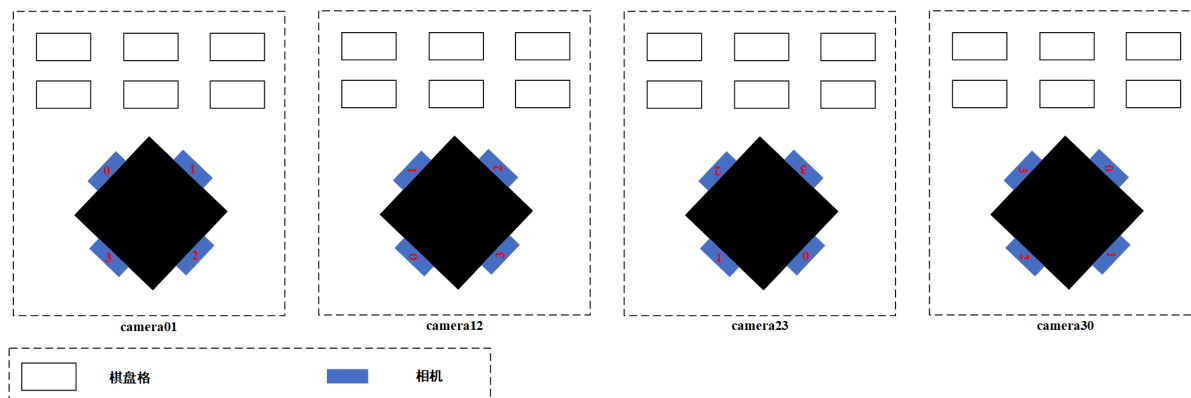


图5-3 便捷模式拍图流程

图像命名规则为：“camera”+ 相机编号 + “” + 两两相机的编号 + “” + 图像序号（3位数，不足3位数的前面补0），例如：

- 0号相机与1号相机拍摄的图为：“camera0_01_000”和“camera1_01_000”

- 2号相机与3号相机拍摄的图为：“camera2_23_000”和“camera3_23_000”
- 3号相机与0号相机拍摄的图为：“camera3_30_000”和“camera0_30_000”

保存格式为：.jpg、.yuv、.bin 等常用格式。

下图为实际拍摄到的图像及其命名实例：



camera0_01_000.bin



camera1_01_000.bin



camera1_12_000.bin



camera2_12_000.bin



camera2_23_000.bin



camera3_23_000.bin



camera3_30_000.bin



camera0_30_000.bin

图5-4 便捷模式示例采图

5.2 快速模式

快速模式与便捷模式不同，需要较为宽敞的标定环境，将多个棋盘格摆放在相机的周边，保证每个重叠带都能拍摄到棋盘格图像。

5.2.1 标定场景

通过图5-5可知，对于四目环形拼接模组的标定，每个重叠带需要两块棋盘格，四个重叠带共需8块棋盘格。

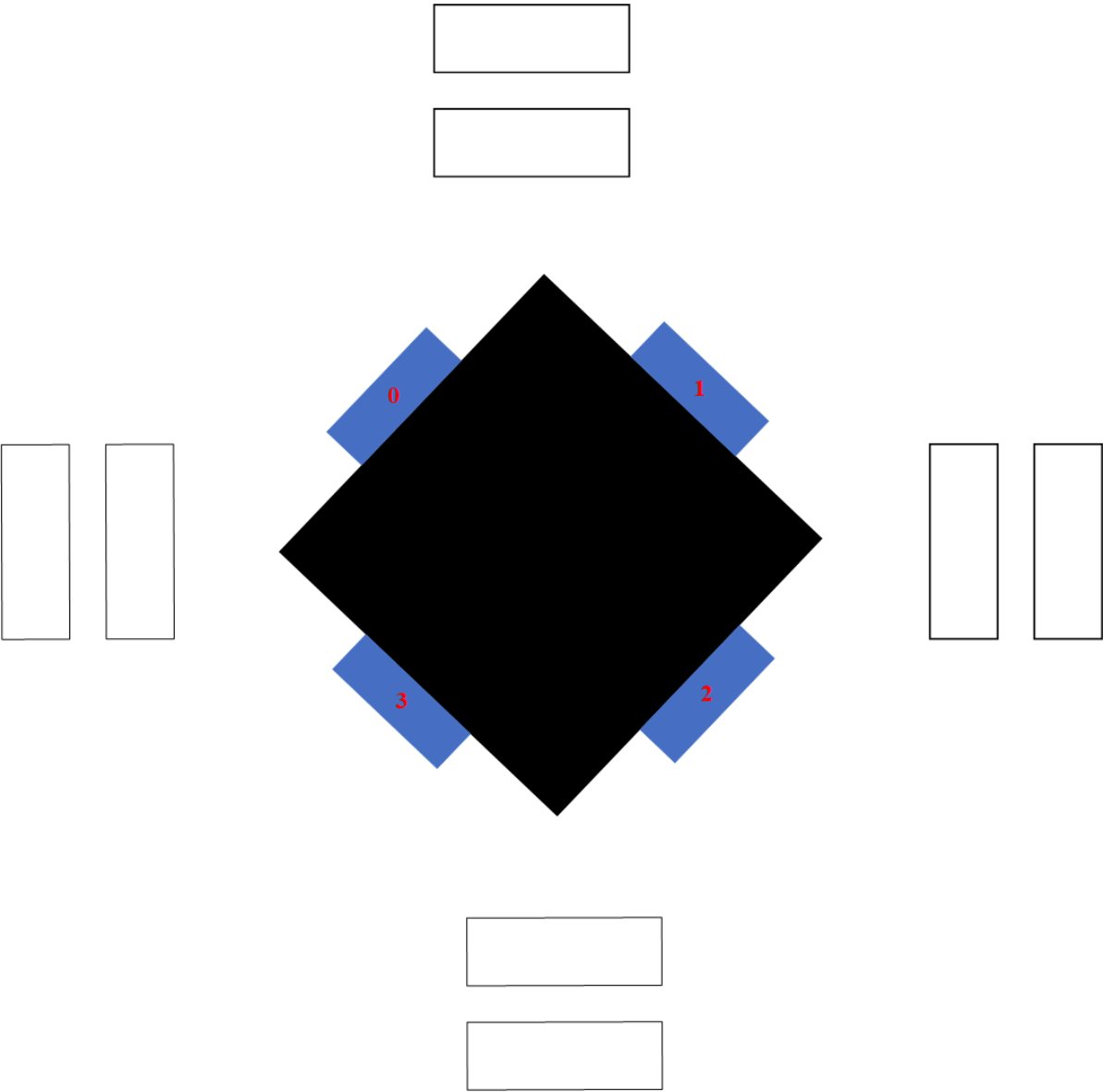


图5-5 标定场景布局示意图

下图为六目非环形拼接场景布局示意图：

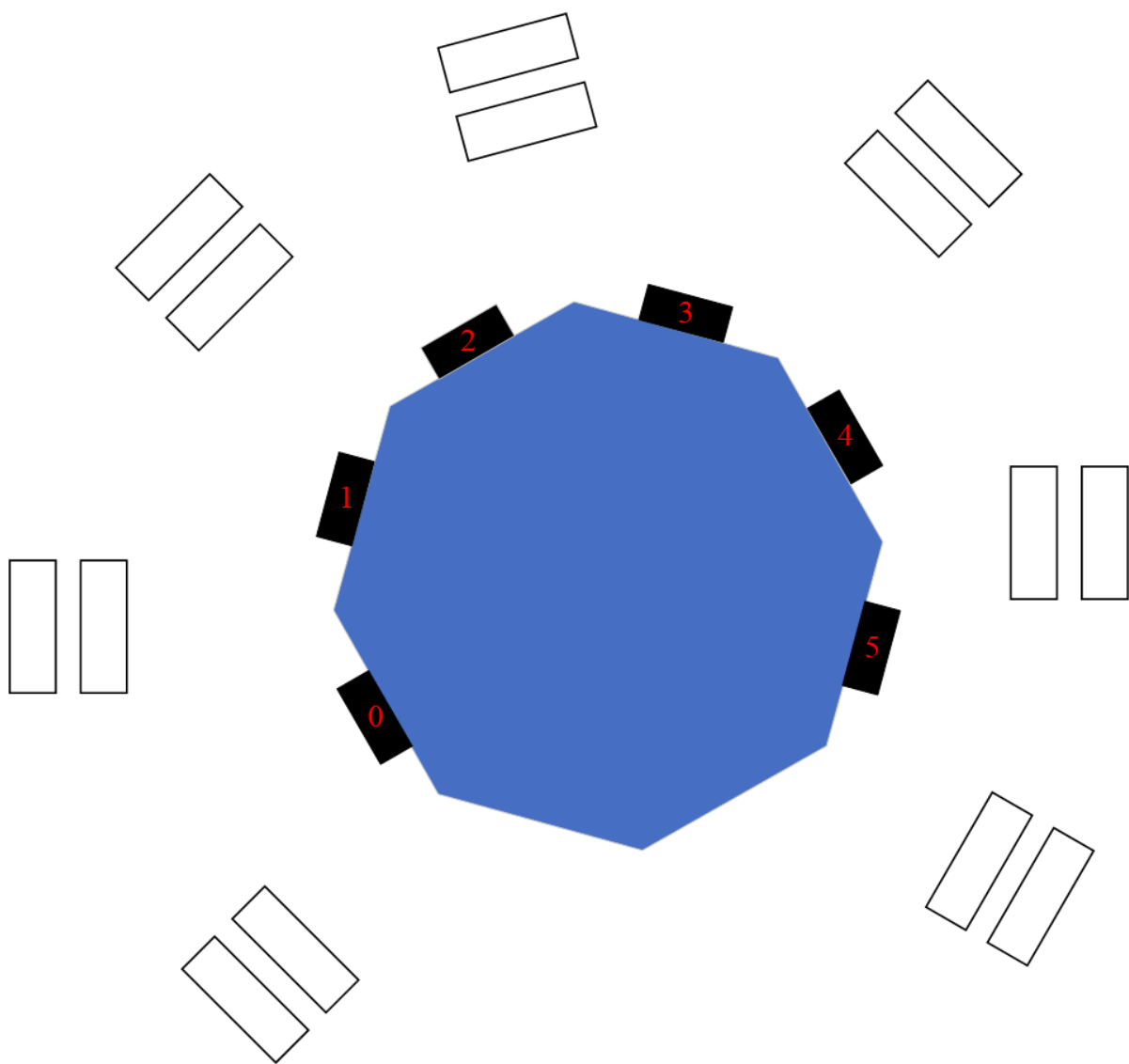


图5-6 非环形拼接场景布局示意图

5.2.2 拍图流程

按照图5-5示意图完成标定环境准备，四个相机同时拍摄一组图像，按照如下规则命名图像：图像命名规则为：“camera”+ 相机编号 + “_” + 图像序号（3位数，不足3位数的前面补0），如图5.5所示，图像名分别为camera0_000，camera1_000，camera2_000，camera3_000，保存为“.jpg”, “.yuv”, “.bin”等常用格式。

下图为实际拍摄到的图像及其命名实例：



camera0_000.bin



camera1_000.bin



camera2_000.bin



camera3_000.bin

图5-7 快速模式示例采图

5.3 API接口及Demo

5.3.1 接口定义

```

/*
 * @enum    rkAVS_MULTI_PRODUCT_CONFIG_S
 * @brief   input params.
 */
typedef struct rkAVS_MULTI_PRODUCT_CONFIG_S
{
    AVS_CALIB_IMAGE_PARAMS_S stImageParams;
    /**<image params*/
    AVS_CALIB_BOARD_PARAMS_S stBoardParams;
    /**<board params*/
    AVS_CALIB_CAMERA_PARAMS_S stCameraParams;
    /**<camera params*/
    uint32_t u32IsRing;
    /**<ring 0-1-2-3-4-0*/
    uint32_t u32IsVerify;
    /**<verify calibration params*/
    uint32_t u32MultiProMode;
    /**<multi product calibration mode*/
    uint32_t u32IsSaveCornersImage;
    /**<is save corners image*/
    const char* pcInputPath;
    /**<calibration images path*/
    const char* pcImageNameFormat;
    /**<image name format .yuv, .png, .jpg, .bmp*/
    const char* pcTorrentPath;
    /**<model calibration result path*/

```



```

isSaveCornersImage: 1 //draw corners and
save
multiProductMode: 0 //multi product mode
isRing: 1 //is ring
isVerify: 0 //1-is verify
mode,0-is calib mode

```

以上是多目产线标定的输入配置文件。

```

//多目产线标定读取配置文件
int readMultiCalibProdictConfig(std::string readPath, std::vector<std::string>&
stringData, std::vector<int32_t>& int32Data)
{
    std::string stringLine;
    std::ifstream ifs(readPath, std::ifstream::in);
    if (ifs.fail())
    {
        std::cout << "read calib product config fail!" << std::endl;
    }
    int32_t lineNum = 0;
    int32_t valueData = 0;
    while (getline(ifs, stringLine))
    {
        std::stringstream ss(stringLine);
        std::string stringUnit;
        std::getline(ss, stringUnit, ' ');
        std::getline(ss, stringUnit, ' ');
        if (lineNum < 4)
        {
            stringData.push_back(stringUnit);
        }
        else
        {
            valueData = atoi(stringUnit.c_str());
            int32Data.push_back(valueData);
        }
        lineNum++;
    }
    ifs.close();
    return 0;
}

```

以上是读取多目产线标定配置文件的代码。

```

int32_t multiProductCalibDemo(std::string configPath)
{
    std::vector<std::string> stringData;
    std::vector<int32_t> int32Data;
    readMultiCalibProdictConfig(configPath, stringData, int32Data);
    std::string input_calib_image_path = stringData[0]; //标定图像所在路径
    std::string output_calib_result_path = stringData[1] + "rk_" +
std::to_string(int32Data[0]) + "_camera_result.xml";
    std::string calib_torrent_path = stringData[2];
    std::string input_image_format = stringData[3]; //标定图像后缀名
}

```

```

//获取版本信息
char avsCalibVersion[128];
rkAVS_getCalibVersion(avsCalibVersion);
//初始化参数
AVS_MULTI_PRODUCT_CONFIG_S stInputParams;
//相机规格
stInputParams.stCameraParams.u32CameraFov = int32Data[2];
stInputParams.stCameraParams.u32CameraNum = int32Data[0];
switch (int32Data[1])
{
case 0:
    stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_PINHOLE;
    break;
case 1:
    stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_OMN;
    break;
case 2:
    stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_FISHEYE;
    break;
default:
    std::cout << "RK_AVS_CALIB: CameraType error!" << std::endl;
    return 1;
}
//棋盘格规格
stInputParams.stBoardParams.u32BoardWidth = int32Data[3];
stInputParams.stBoardParams.u32BoardHeight = int32Data[4];
stInputParams.stBoardParams.u32SquareSize = int32Data[5];
//图像规格
stInputParams.pcImageNameFormat = input_image_format.c_str();
switch (int32Data[9])
{
case 0:
    stInputParams.stImageParams.eImageFormat = RK_AVS_IMAGE_FORMAT_TYPE_GRAY;
    break;
case 1:
    stInputParams.stImageParams.eImageFormat = RK_AVS_IMAGE_FORMAT_TYPE_RGB;
    break;
case 2:
    stInputParams.stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_YUVNV12;
    break;
default:
    std::cout << "RK_AVS_CALIB: ImageFormate error!" << std::endl;
    return 1;
}
stInputParams.stImageParams.u32ImageHeight = int32Data[7];
stInputParams.stImageParams.u32ImageWidth = int32Data[6];
if (stInputParams.stImageParams.eImageFormat == RK_AVS_IMAGE_FORMAT_TYPE_RGB)
    stInputParams.stImageParams.u32ImageStride =
stInputParams.stImageParams.u32ImageWidth * 3;
else
    stInputParams.stImageParams.u32ImageStride = int32Data[8];
//输入输出路径
stInputParams.pcInputPath = input_calib_image_path.c_str();
stInputParams.pcCalibResultPath = output_calib_result_path.c_str();
stInputParams.pcTorrentPath = calib_torrent_path.c_str();

```

```
//功能开关
stInputParams.u32IsSaveCornersImage = int32Data[10];
stInputParams.u32MultiProMode = int32Data[11];
stInputParams.u32IsRing = int32Data[12];
stInputParams.u32IsVerify = int32Data[13];
//多目产线标定
AVS_STEREO_CALIB_CHECK_S stStereoCheckParams;
rkAVS_calibProductMulti(&stInputParams, &stStereoCheckParams);
//保存产线筛选参数
std::string saveCheckPath = stringData[1] + "multiProductCheckParams.txt";
saveCheckParams(saveCheckPath, &stStereoCheckParams);
return 0;
}
```

6. 多目产线验证

多目产线验证目的是提前筛选出问题模组，提高出厂良率。多目产线验证同样可以分为客观评测和主观评测。

6.1 客观评测

标定验证过程主要利用模组标定参数即标定输出的xml文件，对指定验证chart板进行验证，利用检测结果与实际值进行对比，进而筛选出有问题的模组。标定验证主要用于筛选出以下两种问题模组：

- 结构不稳定的模组，完成标定后的一段时间模组结构产生形变，已标定结果不能精准描述模组结构信息，需通过标定验证来筛选出来此类问题模组；
- 标定异常的模组，在标定过程中由于操作人员失误导致标定失败的模组。

6.1.1 便捷模式

该模式的拍摄场景和拍图流程可参考章节5.1中产线标定的场景和拍摄流程。为了简化场景，该模式下可以只保留中间2张棋盘格，棋盘格摆放如图6-1所示，按照章节5.1中描述的流程进行拍摄并命名即可。

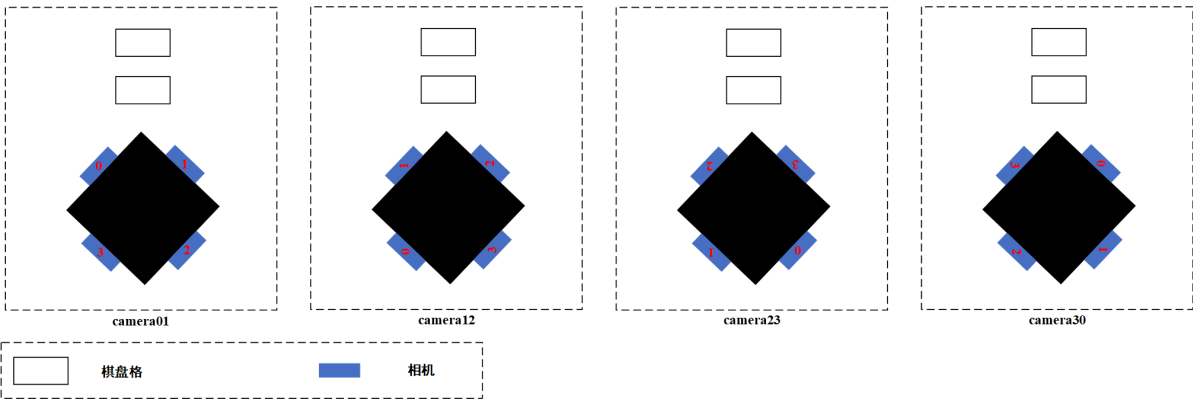


图6-1 产线验证拍摄场景示意图

6.1.2 快速模式

该模式下的环形拼接棋盘格摆放及拍摄流程和章节5.2完全相同，非环形拼接棋盘格摆放可以简化，如图6-2所示。

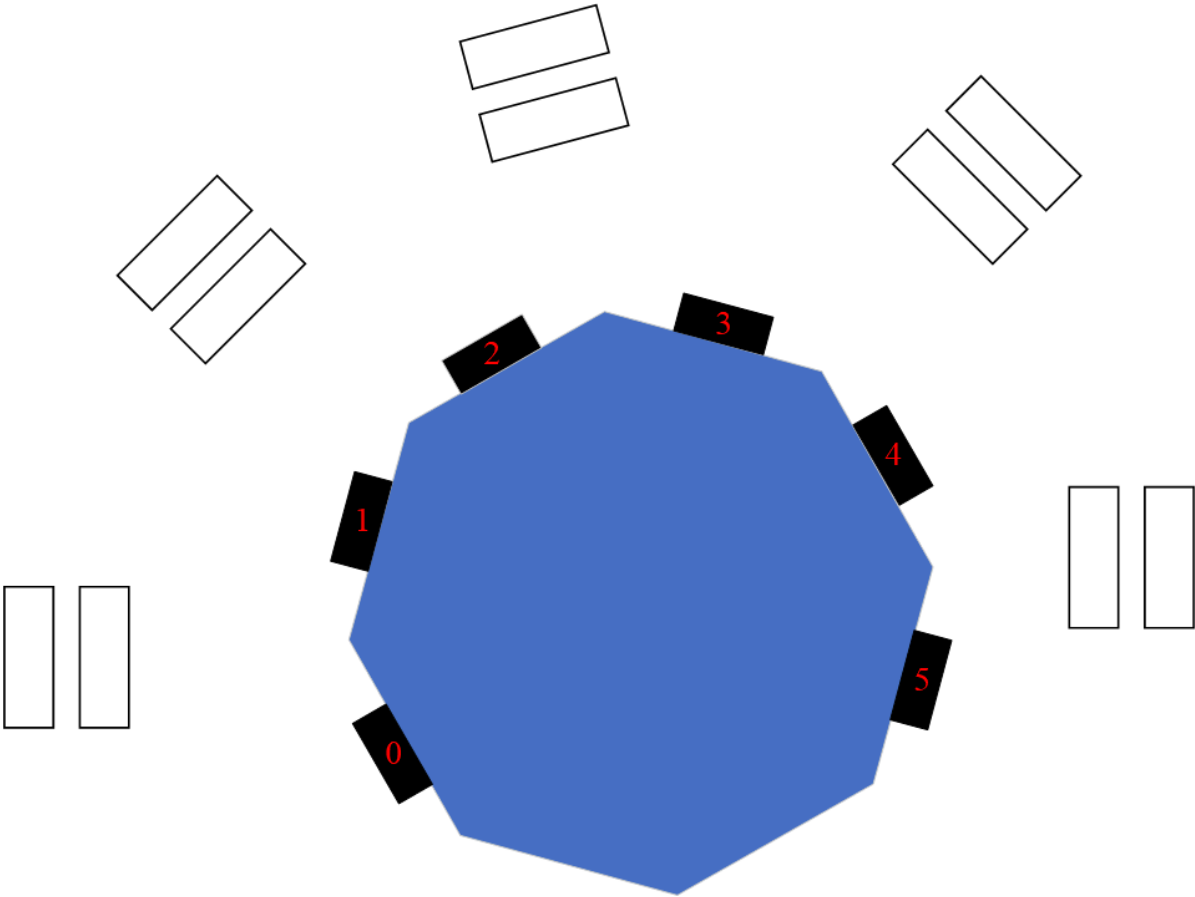


图6-2 产线验证拍摄场景示意图

6.1.3 API接口及Demo

以下是多目产线验证的配置参数及测试demo。

```
srcPath: ./multiVerifyProduct/src/
dstPath: ./multiVerifyProduct/rk_calib_result/
calibTorrentPath: ./multiVerifyProduct/rk_4_camera_result.xml
imageNameFormat: .bin
    // .jpg、.png、.bmp、.bin、.yuv
cameraNum: 4
    // camera number (2-8)
cameraType: 2                                // 0-pinhole、1-omn、
2-fisheye
cameraFov: 120                                // horizontal fov
boardWidth: 15                                // number of
chessboard corners in horizontal direction
boardHeight: 8                                // number of
chessboard corners in vertical direction
squareSize: 30                                // size of
chessboard(mm)
imageWidth: 2560                                // image width
imageHeight: 1520                                // image height
```

```

imageStride: 2560 //image stride
imageFormat: 2 //0-gray、1-rgb、2-yuv
isSaveCornersImage: 1 //draw corners and save
multiProductMode: 0 //multi product mode
isRing: 1 //is ring
isVerify: 1 //1-is verify
mode,0-is calib mode

```

```

int32_t multiProductVerifyDemo(std::string configPath)
{
    std::vector<std::string> stringData;
    std::vector<int32_t> int32Data;
    readMultiCalibProectConfig(configPath, stringData, int32Data);
    std::string input_calib_image_path = stringData[0]; //标定图像所在路径
    std::string output_calib_result_path = stringData[1] + "rk_" +
std::to_string(int32Data[0]) + "_camera_result.xml";
    std::string calib_torrent_path = stringData[1] + "rk_" +
std::to_string(int32Data[0]) + "_camera_result.xml";
    std::string input_image_format = stringData[3]; //标定图像后缀名
    //获取版本信息
    char avsCalibVersion[128];
    rkAVS_getCalibVersion(avsCalibVersion);
    //初始化参数
    AVS_MULTI_PRODUCT_CONFIG_S stInputParams;
    //相机规格
    stInputParams.stCameraParams.u32CameraFov = int32Data[2];
    stInputParams.stCameraParams.u32CameraNum = int32Data[0];
    switch (int32Data[1])
    {
    case 0:
        stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_PINHOLE;
        break;
    case 1:
        stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_OMN;
        break;
    case 2:
        stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_FISHEYE;
        break;
    default:
        std::cout << "RK_AVS_CALIB: CameraType error!" << std::endl;
        return 1;
    }
    //棋盘格规格
    stInputParams.stBoardParams.u32BoardWidth = int32Data[3];
    stInputParams.stBoardParams.u32BoardHeight = int32Data[4];
    stInputParams.stBoardParams.u32SquareSize = int32Data[5];
    //图像规格
    stInputParams.pcImageNameFormat = input_image_format.c_str();
    switch (int32Data[9])
    {
    case 0:
        stInputParams.stImageParams.eImageFormat = RK_AVS_IMAGE_FORMAT_TYPE_GRAY;
        break;

```

```

    case 1:
        stInputParams.stImageParams.eImageFormat = RK_AVS_IMAGE_FORMAT_TYPE_RGB;
        break;
    case 2:
        stInputParams.stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_YUVNV12;
        break;
    default:
        std::cout << "RK_AVS_CALIB: ImageFormat error!" << std::endl;
        return 1;
}
stInputParams.stImageParams.u32ImageHeight = int32Data[7];
stInputParams.stImageParams.u32ImageWidth = int32Data[6];
if (stInputParams.stImageParams.eImageFormat == RK_AVS_IMAGE_FORMAT_TYPE_RGB)
    stInputParams.stImageParams.u32ImageStride =
stInputParams.stImageParams.u32ImageWidth * 3;
else
    stInputParams.stImageParams.u32ImageStride = int32Data[8];
//输入输出路径
stInputParams.pcInputPath = input_calib_image_path.c_str();
stInputParams.pcCalibResultPath = output_calib_result_path.c_str();
stInputParams.pcTorrentPath = calib_torrent_path.c_str();
//功能开关
stInputParams.u32IsSaveCornersImage = int32Data[10];
stInputParams.u32MultiProMode = int32Data[11];
stInputParams.u32IsRing = int32Data[12];
stInputParams.u32IsVerify = int32Data[13];
//产线验证
AVS_STEREO_CALIB_CHECK_S stStereoCheckParams;
rkAVS_calibProductMulti(&stInputParams, &stStereoCheckParams);
//保存产线筛选参数
std::string saveCheckPath = stringData[1] + "multiVerifyCheckParams.txt";
saveCheckParams(saveCheckPath, &stStereoCheckParams);
return 0;
}

```

6.2 主观评测

与双目产线验证主观评测相同，多目产线验证的主观评测也是将标定文件导入板端，运行AVS获取拼接输出图，通过人眼观测拼接图拼缝处的对齐效果进行筛选，主要步骤如下：

- 确认拼接效果验证的环境：光滑的墙面，大尺寸棋盘格（将棋盘格张贴在墙面上）
- 将标定文件导入板端
- 板端运行AVS，并设置最佳拼接距离为相机到墙面的距离
- 调整模组姿态，分别使得每个拼缝落在墙面张贴的棋盘格上
- 观测拼缝处的拼接效果：拼缝处没有显著错位认为标定效果满足需求，如果存在显著错位认为标定效果不佳需要重新标定

7. 标定异常分析

7.1 RK_AVS_CALIB_STATUS_INVALID_PARAM

算法库返回异常值3，表示输入参数有问题，请检查图像输入尺寸、棋盘格参数及其他配置参数是否符合要求。

7.2 RK_AVS_CALIB_STATUS_EOF

算法库返回异常值-1，表示算法库内部计算错误，主要分为以下几种情况。

7.2.1 Split chessboard error

输入单帧图像内包含多个棋盘格，内部标定时需要区分多个棋盘格所在位置，该错误表明算法检查多个棋盘格时出现错误。遇到该问题可通过以下几种方式解决：

- 尽可能保证拍摄背景为纯白或者纯黑（纹理较少），如在背景为白墙的位置拍摄标定板或者在标定板后面布置一块白色或者黑色的幕布、板子；
- 通过调节ISP保证输入图像整体亮度均匀，不可存在局部过曝、过暗等情况。

7.2.2 Good chessboard number < n

输入的图像内包含多个棋盘格，如果内部算法检测到的棋盘格个数少于指定棋盘格个数，算法会报该错误并返回-1。遇到该问题可通过以下几种方式解决：

- 保证所需棋盘格（标定每个相机为4个棋盘格，验证为2个棋盘格）都在相机视野内，不可存在拍摄不完整的棋盘格；
- 调节ISP保证输入图像整体亮度均匀，不可存在局部过曝、过暗的情况,如图7-1所示；
- 保证多个棋盘格之间有足够的距离；
- 拍摄棋盘格图像清晰，不能出现某个棋盘格模糊的情况，如图7-2右侧两个棋盘格所示；
- 四个棋盘格应分布在对应的辅助框内，如图7-3所示。

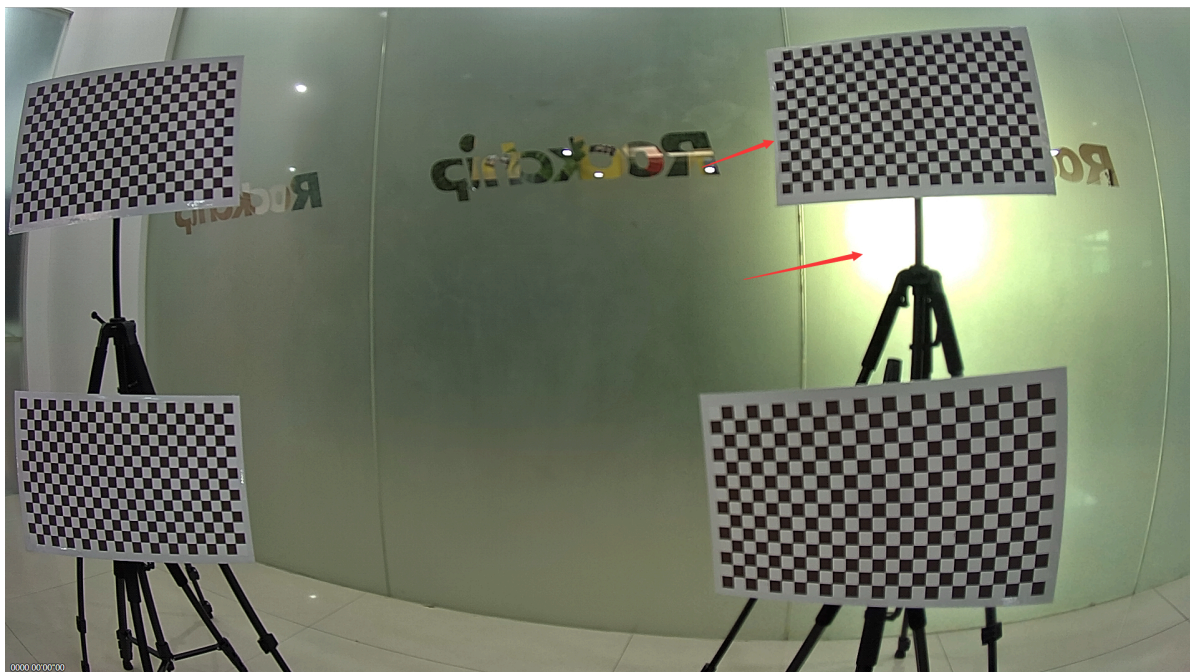


图7.1 局部过曝模糊示例



图7-2 棋盘格模糊示例

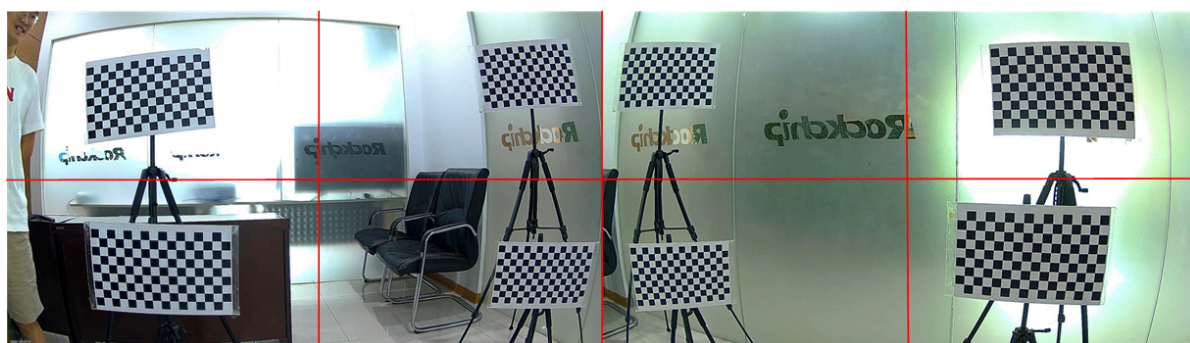


图7-3 棋盘格摆放位置说明

通过检查保存的角点图判断哪个棋盘格没有检测到，如图7-4所示，右上角的棋盘格因为周边过曝，导致角点提取失败。



图7-4 棋盘格角点提取图

7.3 标定精度低

7.3.1 问题分析

该问题主要体现在标定输出的RMS较大，可通过调节标定板摆放位置优化标定结果：

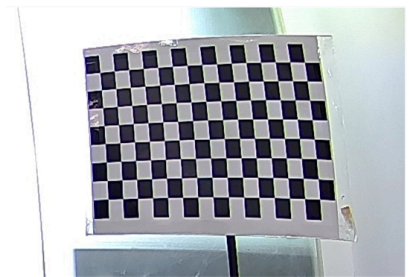
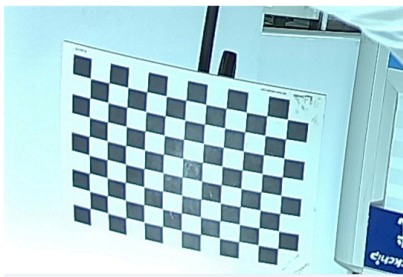
- 按照章节3.2中的场景布局布置棋盘格，保证每个棋盘格角点能够拍摄清晰；
- 查看图像是否存在反光、过曝、过暗、噪点多的情况，改善图像质量；
- 重叠区域的两个棋盘格上下不对称可能会影响精度；
- 可采用背光板提高标定精度。



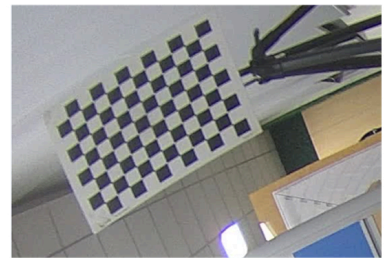
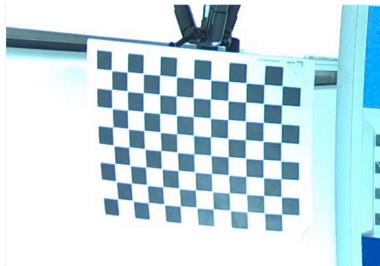
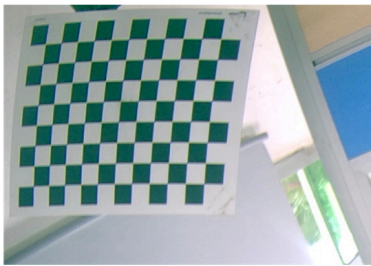
图7-1 标定板拍摄示例图

7.3.2 错误示例

- 反光



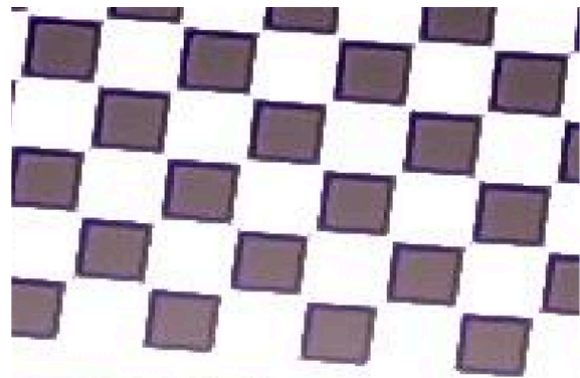
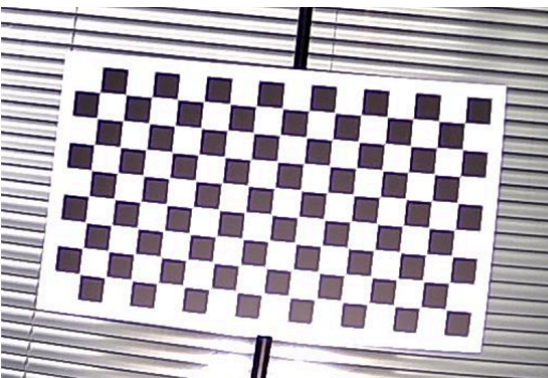
- 棋盘格模糊



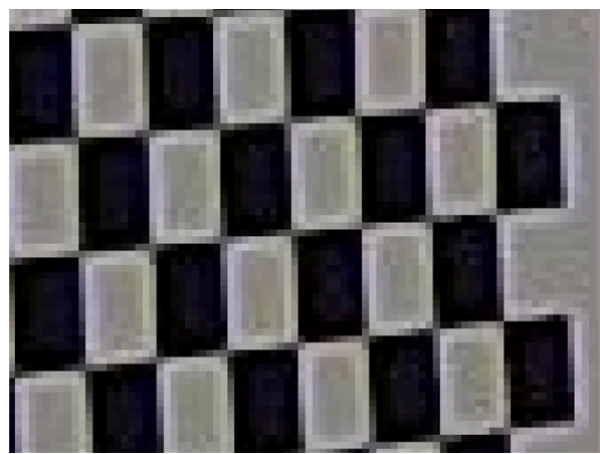
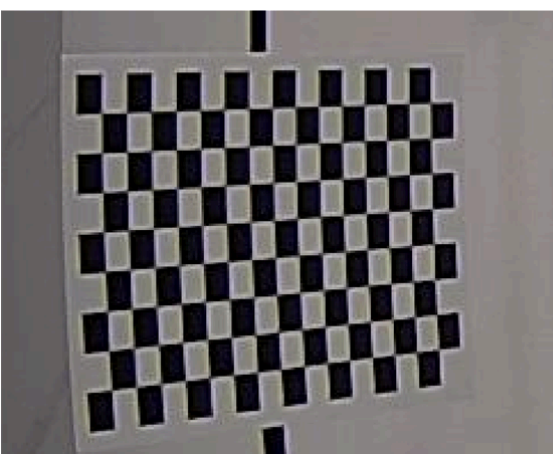
- 棋盘格不平整

张贴棋盘格不平整，存在鼓包等现象。

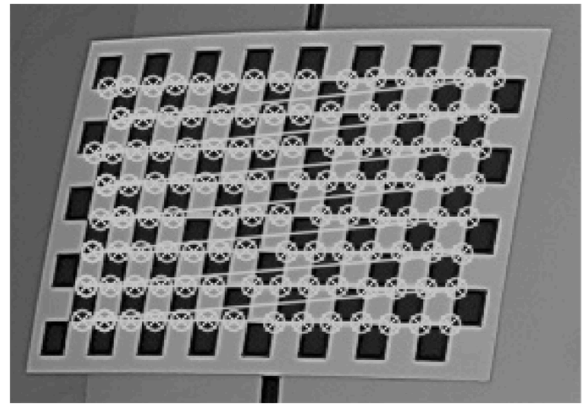
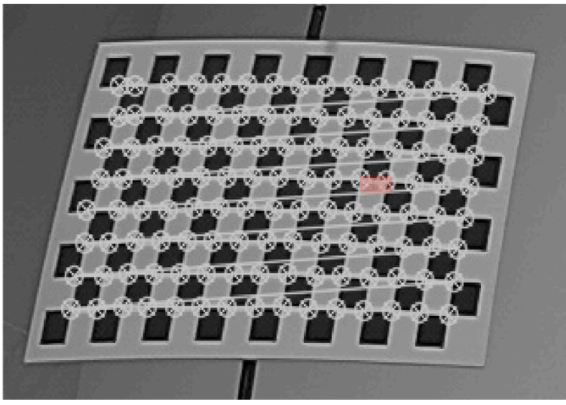
- 棋盘格过曝



- 光照不足

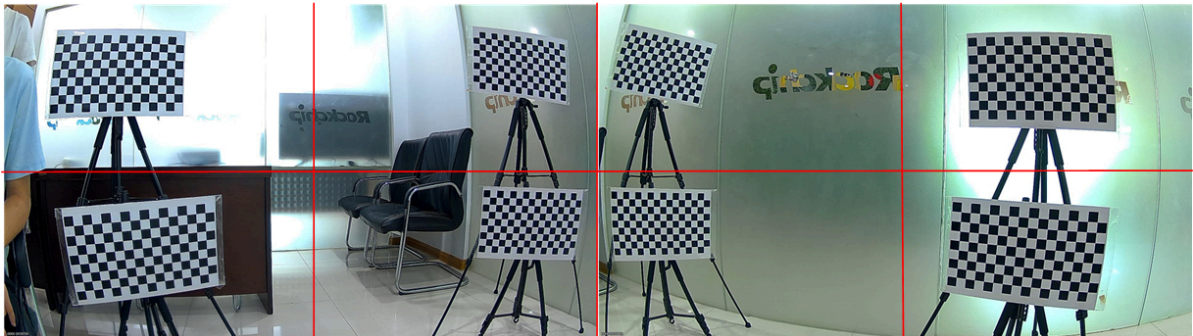


- 角点提取出错

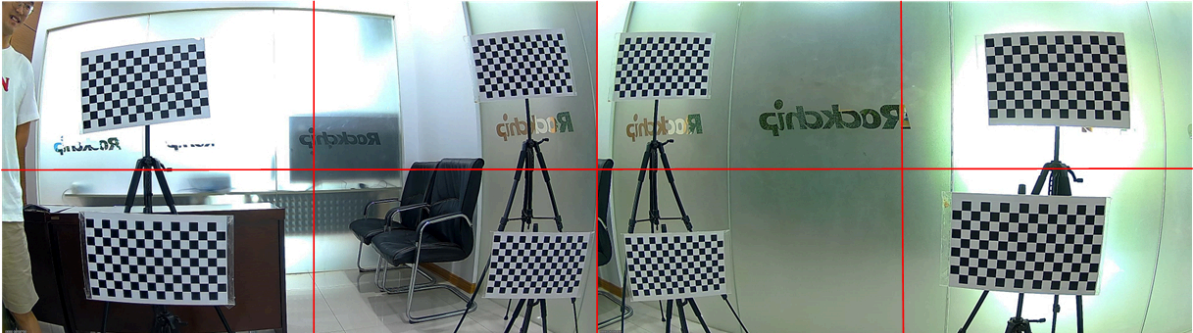


如上图所示，棋盘格角点提取出错，这时需要分析角点提取出错的原因，一般有以下几种情况：棋盘格过曝、光照条件差、棋盘格尺寸占据视野面积太小这三种原因。

- 重叠区域棋盘格上下不对称



如上图所示，中间重叠区域的两个棋盘格相对于中线不够上下对称，也会影响标定精度，最好可以像下图所示进行摆放。



- 拍摄时相机或者棋盘格存在震动